PHASE G · JALON D · COURS · 25 OCTOBRE 2026 · VERSION 2

Ne pas rastériser : la hiérarchie d'Abrash, pas à pas

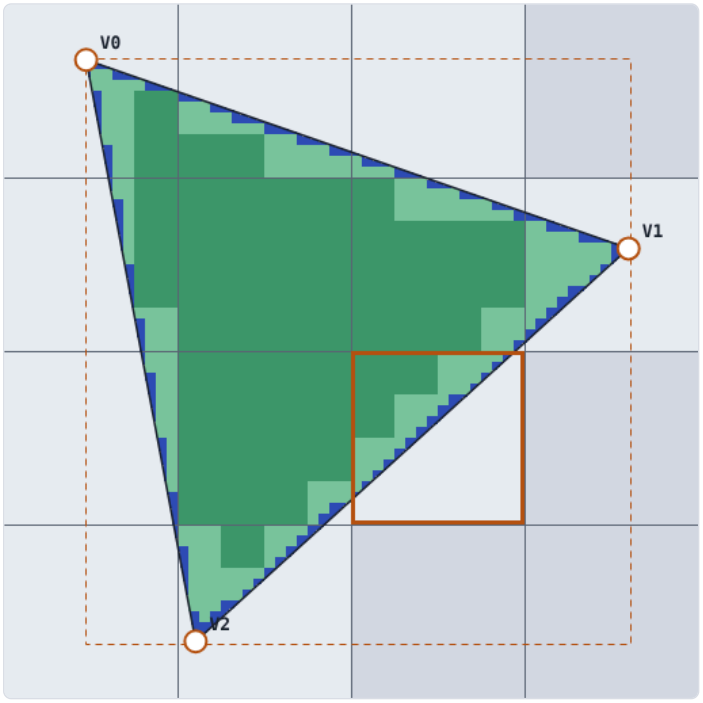
En 2009, pour Larrabee, Michael Abrash a écrit un rastériseur logiciel qui gagne surtout en évitant le travail. Il ne cherche pas à tester les pixels plus vite. Il cherche à en tester le moins possible : une tuile entière est jugée d'un coup, ses 16 blocs sont jugés en une instruction, et un bloc entièrement couvert est dessiné sans un seul test.

Ce cours suit l'ordre de son article. La **partie A** construit la méthode une idée à la fois, chaque idée avec sa figure animée : l'équation d'arête, le triangle comme intersection de trois demi-plans, les coins de rejet et d'acceptation, la descente, les masques. La **partie B** traite ce que l'article survole : rendre tout cela exact au bit près, et ce que cela veut dire pour LV3. Le code est replié sous chaque chapitre : on le lit une fois l'idée comprise.

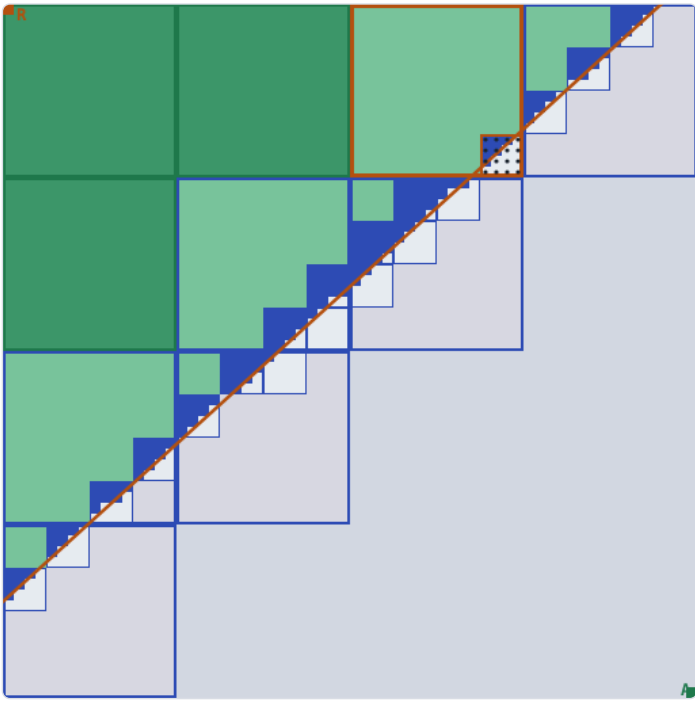
Statut : cours, pas un lot à appliquer. Le jalon D est rangé ; il s'ouvrira en D1 à la clôture du chantier C si la règle écrite d'avance le déclenche. Source : M. Abrash, *Rasterization on Larrabee*, Dr. Dobb's, mai 2009.

LABO Le laboratoire : votre triangle, vos tuiles

Toutes les figures du cours travaillent sur le même triangle et la même sélection. Choisissez-les ici. À gauche, la cible de 256 × 256 pixels découpée en 16 tuiles de 64 × 64 ; faites glisser un sommet, touchez une tuile. À droite, la tuile choisie ; touchez un bloc pour l'étudier dans les chapitres A10 à A12.



Tuiles de la boîte : 16 · rejetées 4 · pleines 1 · partielles 11 · tuile choisie (2, 2)



Tuile (2, 2) partielle · bloc 16×16 n° 2 partiel · bloc 4×4 n° 15 partiel · R / A : coins de l'arête e0

tuile ou bloc 16×16 plein · bloc 4×4 plein · pixel produit par un masque · rejeté · bloc partiel (on descend) · faux positif · sélection

20 053 pixels couverts	**identique** hiérarchie = force brute	**94,0 %** pixels émis par blocs pleins	**38 / 50** blocs 16×16 pleins / partiels
314 / 154 blocs 4×4 pleins / partiels	**1** faux positifs (4×4 vides)	**6 500** instr. couverture (hiérarchie)	**207 826** instr. couverture (C2)

PARTIE A · LA MÉTHODE, UNE IDÉE À LA FOIS

A1 L'équation d'arête : un nombre qui dit de quel côté

Une arête est un segment, mais pour le rastériseur c'est une droite qui coupe le plan en deux. À chaque point on associe un nombre, la valeur de l'équation d'arête. Elle vaut zéro sur la droite, elle est positive

d'un côté, négative de l'autre, et elle grandit à mesure qu'on s'éloigne. Tout le reste de la méthode repose sur ce nombre.

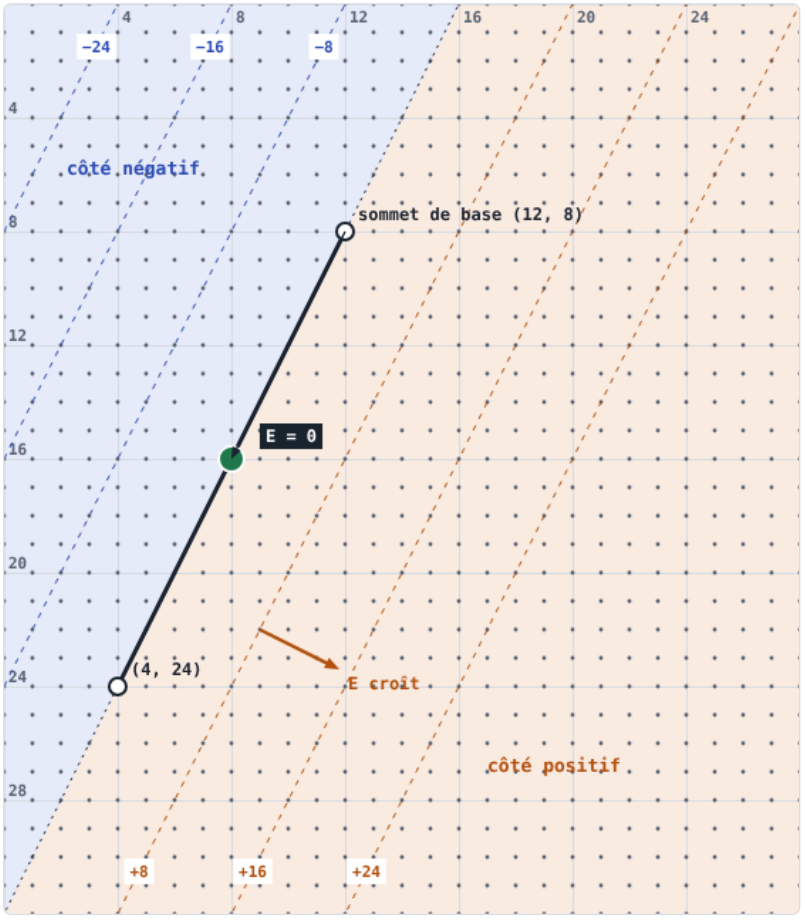


Figure A1. L'exemple de l'article : l'arête va de (12, 8) à (4, 24). Les lignes pointillées relient les points de même valeur. Touchez un point de la grille.

Points de l'article :

```
Point (8, 16), mesuré depuis le sommet de base
(12, 8) :
  x' = 8 − 12 = -4      y' = 16 − 8 = 8

Équation d'Abrash : B·x' + C·y'
  B = y1 − y0 = 24 − 8 = 16
  C = x0 − x1 = 12 − 4 = 8
  16·(-4) + 8·(8) = 0
Divisée par 8 (même signe) :  2·x' + y' = -8 + 8 =
0

→ zéro : le point est SUR l'arête
Un pas à droite : +2. Un pas vers le bas : +1.
Partout.
```

À OBSERVER

- Sur l'arête, la valeur est 0, quel que soit le point.
- Un pas d'une case vers la droite ajoute toujours la même quantité (A), un pas vers le bas aussi (B). C'est une fonction *affine* : le pas ne dépend pas de l'endroit.
- Les lignes de niveau sont parallèles à l'arête et régulièrement espacées : la valeur mesure une distance à la droite, à un facteur près.

Abrash écrit l'équation sous la forme **B·x + C·y**, avec B = y1 − y0 (la hauteur de l'arête) et C = x0 − x1 (moins sa largeur), x et y étant mesurés depuis un sommet de l'arête, où la valeur est connue : zéro. Pour son exemple, cela donne 16·x + 8·y, qu'on peut diviser par 8 sans changer aucun signe : **2x + y**. Dans la suite du cours, on écrit E = A·x + B·y + C, comme dans le code (le A du cours est le B d'Abrash).

SCALAIRE

```
// E(P) = A·Px + B·Py + C, P en sous-pixels. Convention : E < 0 à l'intérieur.
struct EdgeEq { int32_t A, B; int64_t C; };

// Aire signée ×2 : (V1 − V0) × (V2 − V0). En 64 bits : produit de deux écarts de coordonnées.
const int64_t area2 = int64_t(X1 - X0) * (Y2 - Y0) - int64_t(Y1 - Y0) * (X2 - X0);
if (area2 == 0) return;                    // dégénéré : aucun pixel
const int32_t sg = (area2 > 0) ? -1 : 1;   // retourne le signe pour que l'intérieur soit < 0
                                           // (dans LV3, l'orientation est déjà connue : B4/B5)

// Arête a -> b :  (b − a) × (P − a) = (a.y − b.y)·Px + (b.x − a.x)·Py + cste
auto MakeEdge = [sg](int32_t ax, int32_t ay, int32_t bx, int32_t by) noexcept
{
    EdgeEq e;
    e.A = sg * (ay - by);                                  // pas de E pour +1 sous-pixel en x
    e.B = sg * (bx - ax);                                  // pas de E pour +1 sous-pixel en y
    e.C = -(int64_t(e.A) * ax + int64_t(e.B) * ay);        // E(a) = 0 : la droite passe par a
    return e;
};
const EdgeEq e0 = MakeEdge(X1, Y1, X2, Y2);   // opposée au sommet 0 (comme w0 de LV3)
const EdgeEq e1 = MakeEdge(X2, Y2, X0, Y0);
const EdgeEq e2 = MakeEdge(X0, Y0, X1, Y1);
```

AVX2 : 8 TRIANGLES EN SOA

```
// 8 triangles à la fois, comme ClassifyFaces8 (B4) : Xa8/Ya8 = sommet a des 8 triangles, etc.
// sg8 : +1 ou −1 par voie (orientation). _mm256_sign_epi32(v, s) : v si s > 0, −v si s < 0.
const __m256i A8 = _mm256_sign_epi32(_mm256_sub_epi32(Ya8, Yb8), sg8);   // sg·(a.y − b.y)
const __m256i B8 = _mm256_sign_epi32(_mm256_sub_epi32(Xb8, Xa8), sg8);   // sg·(b.x − a.x)

// C = −(A·a.x + B·a.y) déborde 32 bits en général (produit de deux coordonnées).
```

```
// _mm256_mul_epi32 fait 4 produits 32×32 → 64 bits (voies paires) : deux passes par lot de 8.
// En pratique, le setup se fait une fois par TRIANGLE : il reste scalaire dans ce cours,
// seules les tables (étape 8) et la descente (étapes 9 à 12) sont vectorisées.
```

A2 Trois arêtes, un triangle

Un triangle est l'intersection de trois demi-plans. On oriente les trois équations pour que l'intérieur soit toujours du côté négatif. Un centre de pixel est dans le triangle si, et seulement si, ses trois valeurs sont négatives. Il n'y a rien d'autre à tester.

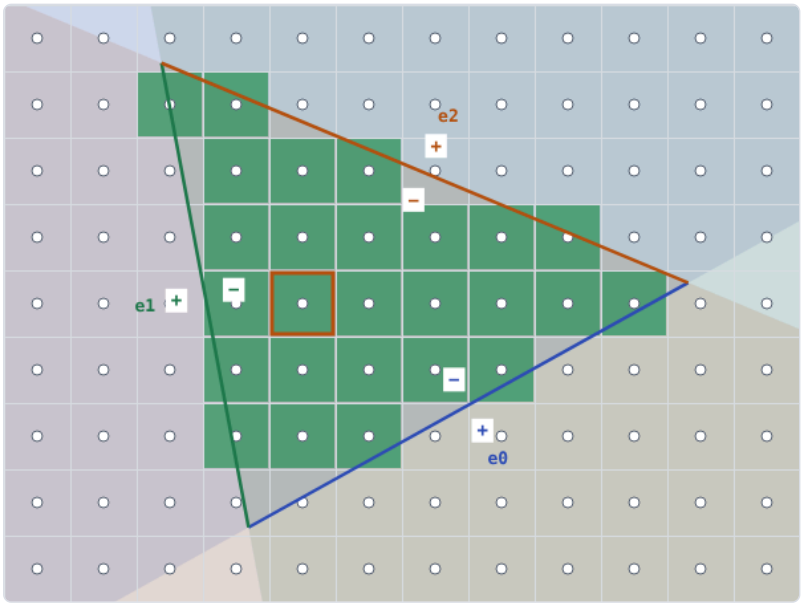


Figure A2. Chaque centre de pixel est un point blanc. En vert : négatif pour les trois arêtes. Affichez les demi-plans un par un pour voir l'intersection se former.

Demi-plan négatif :

```
Pixel (4, 4), centre (4,5 ; 4,5) :
  e0 =   −310  négatif : du bon côté
  e1 =   −162  négatif : du bon côté
  e2 =   −348  négatif : du bon côté

→ les trois sont négatives : le pixel est DANS le triangle
(valeurs en pixels entiers, forme E' de la partie B)
```

À OBSERVER

- Plus on s'enfonce dans le triangle, plus les valeurs sont négatives ; plus on s'en éloigne, plus elles sont positives.
- Un pixel juste dehors n'est en faute que pour une arête : les deux autres sont satisfaites.
- On sait tester un pixel. La vraie question est : *lesquels* tester, sans en tester trop ? C'est le chapitre suivant.

On calcule l'aire signée du triangle ; si elle est positive, on change le signe des trois équations. Les détails d'exactitude (entiers, règle top-left) sont en partie B.

SCALAIRE

```
// E(P) = A·Px + B·Py + C, P en sous-pixels. Convention : E < 0 à l'intérieur.
struct EdgeEq { int32_t A, B; int64_t C; };

// Aire signée ×2 : (V1 − V0) × (V2 − V0). En 64 bits : produit de deux écarts de coordonnées.
const int64_t area2 = int64_t(X1 - X0) * (Y2 - Y0) - int64_t(Y1 - Y0) * (X2 - X0);
if (area2 == 0) return;                    // dégénéré : aucun pixel
const int32_t sg = (area2 > 0) ? -1 : 1;   // retourne le signe pour que l'intérieur soit < 0
                                           // (dans LV3, l'orientation est déjà connue : B4/B5)

// Arête a -> b :  (b − a) × (P − a) = (a.y − b.y)·Px + (b.x − a.x)·Py + cste
auto MakeEdge = [sg](int32_t ax, int32_t ay, int32_t bx, int32_t by) noexcept
{
    EdgeEq e;
    e.A = sg * (ay - by);                                  // pas de E pour +1 sous-pixel en x
    e.B = sg * (bx - ax);                                  // pas de E pour +1 sous-pixel en y
    e.C = -(int64_t(e.A) * ax + int64_t(e.B) * ay);        // E(a) = 0 : la droite passe par a
    return e;
};
const EdgeEq e0 = MakeEdge(X1, Y1, X2, Y2);   // opposée au sommet 0 (comme w0 de LV3)
const EdgeEq e1 = MakeEdge(X2, Y2, X0, Y0);
const EdgeEq e2 = MakeEdge(X0, Y0, X1, Y1);
```

DÉMONSTRATION DU SIGNE

```
E(centre) = 16·(A·x + B·y + q) + r,   avec 0 <= r < 16

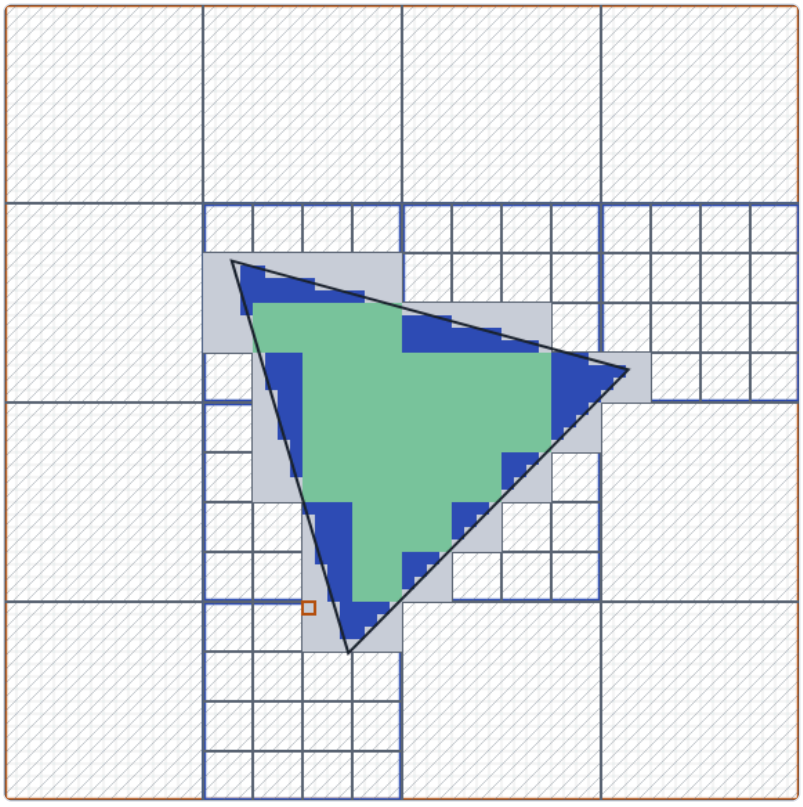
Si A·x + B·y + q <= −1 :
    E <= −16 + r <= −16 + 15 = −1 < 0        → dedans

Si A·x + B·y + q >= 0 :
    E >= 0 + r >= 0                          → dehors

Donc signe(E) = signe(E'), pour TOUT centre de pixel.
Le reste r disparaît : il est le même partout.
```

A3 Comment parcourir ? Cinq stratégies sur le même triangle

Tester un pixel est facile. Le coût vient du nombre de pixels testés pour rien, et des décisions à prendre pour savoir où aller ensuite. Abrash compare trois approches ; on y ajoute les deux boucles actuelles de LV3. Lancez l'animation pour chacune et comparez les compteurs.



```
Hiérarchie (Abrash) · pas 14 / 14

Éléments jugés     : 112 blocs (7 instr.) + 336 pixels
Pixels testés pour rien : 201
Décisions de parcours   : 47
Vectorisable       : oui, 16 voies
Pixels couverts    : 455
```

Stratégie	Jugés	Testés pour rien	Décisions	Vecteurs
Trapèzes (Pixomatic)	30 lignes	0	30	non
Boîte, pixel par pixel (C1)	1089 pixels	634	1089	non
Boîte, blocs de 8 (C2)	165 blocs × 8 voies	865	165	oui, 8 voies
Balayage 4×4	58 tampons × 16	473	58	oui, 16 voies

Figure A3. Une tuile de 64 × 64 et un triangle d'une trentaine de pixels. Bleu : couvert. Gris : testé pour rien. Vert : bloc déclaré plein sans test.

Stratégie	Jugés	Testés pour rien	Décisions	Vecteurs
Hiérarchie (Abrash)	112 blocs (7 instr.) + 336 pixels	201	47	oui, 16 voies

À OBSERVER

- **Trapèzes (Pixomatic 1)** : aucun pixel testé pour rien, mais on avance ligne par ligne en suivant deux arêtes. Impossible à vectoriser, et des spans d'un pixel de haut ne conviennent pas au shading par blocs.
- **Boîte, pixel par pixel (LV3 C1)** et **boîte, blocs de 8 (LV3 C2)** : simples, sans décision de parcours, mais tout le rectangle englobant est testé. Pour un triangle allongé en diagonale, c'est surtout du vide.
- **Balayage 4×4** : un tampon de 16 pixels qu'on promène à gauche, à droite, puis vers le bas. Vectorisable, mais chaque pas est une décision mal prédite, et il faut sortir du triangle pour savoir qu'on en est sorti.
- **Hiérarchie (Abrash)** : une instruction juge 16 blocs à la fois ; on ne descend que dans les blocs à cheval sur une arête. Peu de décisions, et l'intérieur est déclaré plein sans être testé.

A4 Les tuiles : découper la cible avant de rastériser

Larrabee rend la cible par **tuiles** (on dit aussi « chunks » ou « bins »). Chaque triangle est d'abord rangé dans le bac de chaque tuile qu'il touche ; puis chaque tuile est rastérisée seule, avec ses tampons de couleur et de profondeur dans le cache. La rastérisation se fait donc en deux temps : trouver les tuiles touchées, puis rastériser le triangle dans chacune.

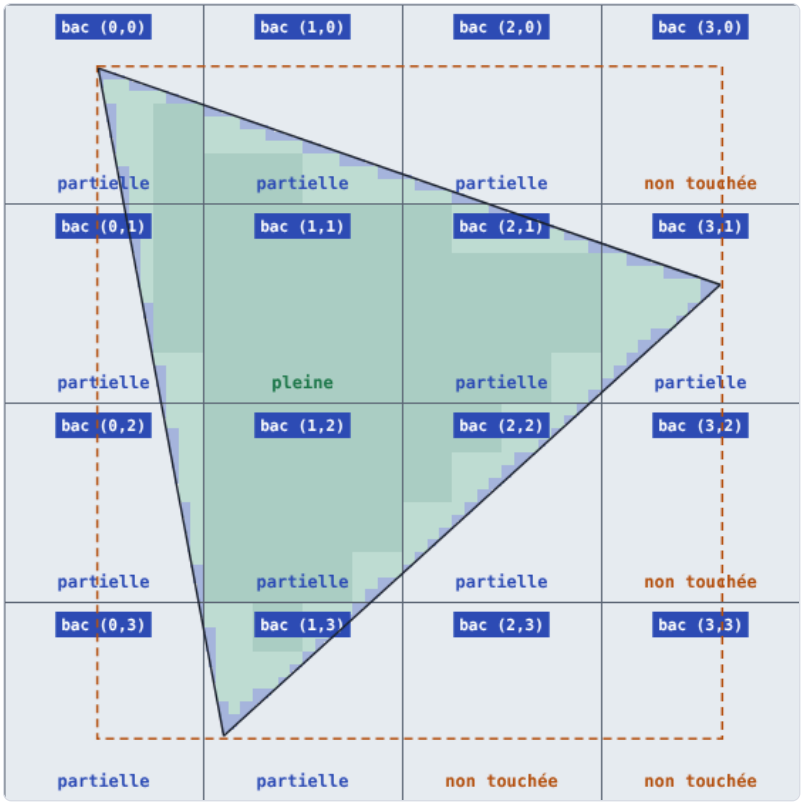


Figure A4. La boîte englobante (pointillés) donne les tuiles candidates. Pour chacune, il reste à savoir si le triangle la touche vraiment : c'est l'objet des deux chapitres suivants.

```
Boîte englobante : x 30 → 230, y 20 → 235 (pixels)
Tuiles candidates (boîte) : 16 sur 16
  · réellement touchées, partielles : 11
  · entièrement couvertes           : 1
  · dans la boîte mais non touchées : 4

Le triangle est rangé dans le bac des tuiles touchées,
avec, pour chacune, ses arêtes actives (chapitre A7).
```

À OBSERVER

- 90 % des triangles d'une scène tiennent dans une tuile : un test de boîte suffit pour eux.
- Pour les grands triangles, la boîte surestime : les tuiles dans les coins de la boîte ne sont pas touchées.
- Pour LV3, les tuiles préparent le multithreading (L13) : un thread par tuile, sans verrou, 32 Ko de tampons par tuile.

A5 Le coin de rejet trivial

Pour savoir si une tuile est entièrement du mauvais côté d'une arête, il suffit de regarder **un seul point** : le coin de la tuile où l'équation est la plus négative, le plus « à l'intérieur ». Si même ce coin-là est dehors (valeur ≥ 0), aucun point de la tuile ne peut être dedans, et le triangle entier est ignoré pour cette tuile.

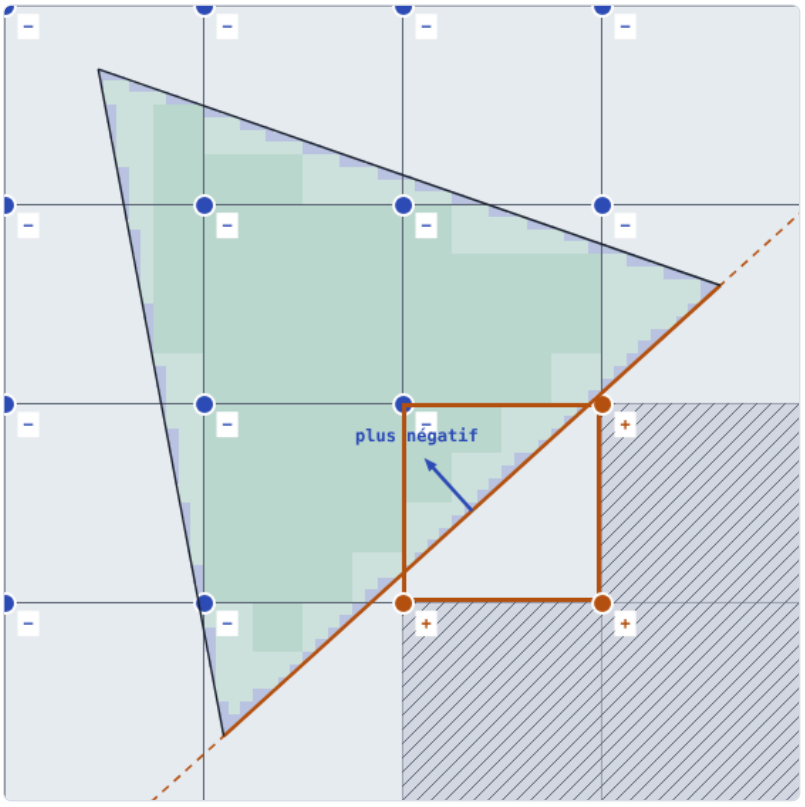
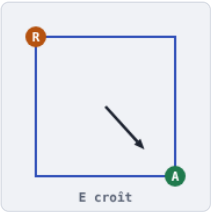


Figure A5. L'arête choisie est prolongée (pointillés). Un point par tuile : son coin de rejet, bleu si la valeur y est négative, orange sinon. Tuiles hachurées : rejetées par cette arête.

Arête :




```
e0 : A = 2 319, B = 2 555
A > 0 : E croît vers la droite → minimum à GAUCHE
B > 0 : E croît vers le bas → minimum en HAUT
R = coin de rejet (minimum), A = coin d'acceptation (maximum), toujours opposés.
```

À OBSERVER

- Le coin de rejet est le même pour toutes les tuiles : il ne dépend que de l'orientation de l'arête, donc des signes de A et B.
- Abrash : « si vous comprenez ceci, vous avez compris 90 % du rastériseur ». Tout le reste applique ce test à des blocs plus petits.
- C'est le test du vertex positif d'une AABB contre un plan, celui du frustum culling de LV3, en 2D.

```
Tuile choisie (2, 2), arête e0 : premier pixel (128, 128), E' = −138 360.
Coin de rejet = pixel (128, 128) : E' = −138 360 + (0) = −138 360  →  < 0 : au moins un pixel peut être du bon côté ; on garde la tuile
Tuiles rejetées par e0 : 3 sur 16.
```

SCALAIRE

```
// Bloc de S×S pixels, premier pixel (x0, y0), où l'arête vaut e = E'(x0, y0).
// Centres : x0 .. x0 + S − 1, y0 .. y0 + S − 1. E' affine → extrêmes aux coins.
struct CornerOffsets { int32_t rej, acc; };

constexpr CornerOffsets Corners(int32_t A, int32_t B, int32_t S) noexcept
{
    const int32_t a = A * (S - 1);        // écart de E' entre la 1re et la dernière colonne
    const int32_t b = B * (S - 1);        // idem entre la 1re et la dernière ligne
    return { std::min(0, a) + std::min(0, b),     // coin de REJET : le plus négatif (le plus dedans)
             std::max(0, a) + std::max(0, b) };   // coin d'ACCEPTATION : le plus positif
}
// rejet  : e + rej >= 0  → aucun centre du bloc n'est dedans : bloc rejeté pour le TRIANGLE
// accept : e + acc <  0  → tous les centres sont dedans : ARÊTE oubliée pour ce bloc
```

AVX2 : 8 ARÊTES

```
// Pour 8 triangles (ou 8 arêtes) à la fois : min/max entiers, sans branche.
const __m256i sm1 = _mm256_set1_epi32(S - 1);
const __m256i a   = _mm256_mullo_epi32(A8, sm1);          // A·(S − 1)
const __m256i b   = _mm256_mullo_epi32(B8, sm1);          // B·(S − 1)
const __m256i z   = _mm256_setzero_si256();
const __m256i rej = _mm256_add_epi32(_mm256_min_epi32(z, a), _mm256_min_epi32(z, b));
const __m256i acc = _mm256_add_epi32(_mm256_max_epi32(z, a), _mm256_max_epi32(z, b));
// vpmulld coûte ≈ 10 cycles de latence : c'est du setup, une fois par triangle.
```

A6 Le coin d'acceptation triviale

Le test inverse prend le coin opposé, celui où l'équation est la plus positive, le plus « à l'extérieur ». Si même ce coin-là est dedans (valeur < 0), toute la tuile est du bon côté de cette arête : on n'aura plus besoin de la tester à l'intérieur de la tuile.

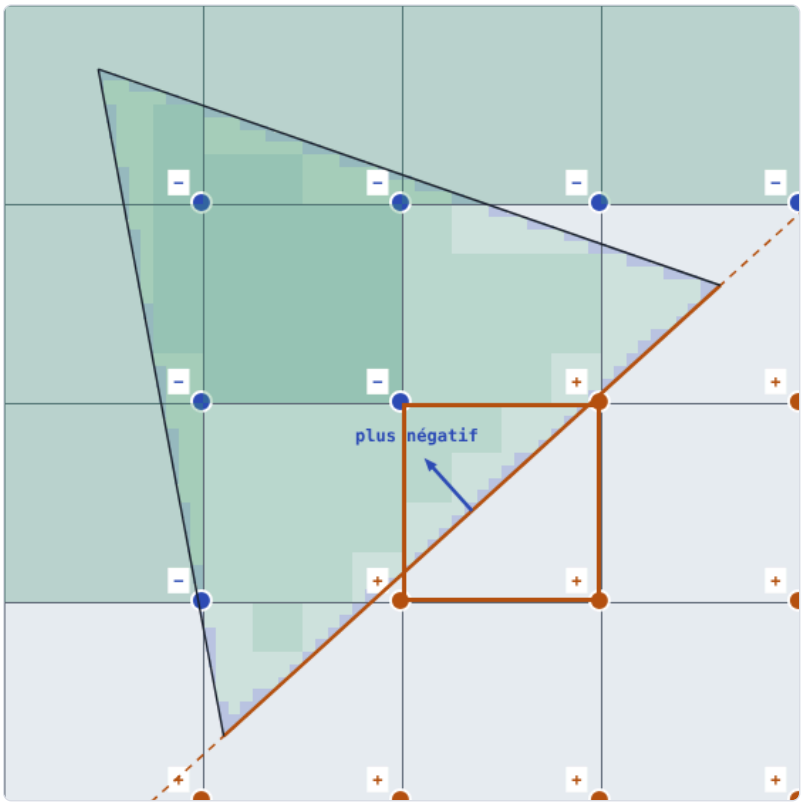


Figure A6. Un point par tuile : son coin d'acceptation. Tuiles teintées en vert : entièrement du bon côté de l'arête choisie. Le coin d'acceptation est toujours à l'opposé du coin de rejet.

Arête :

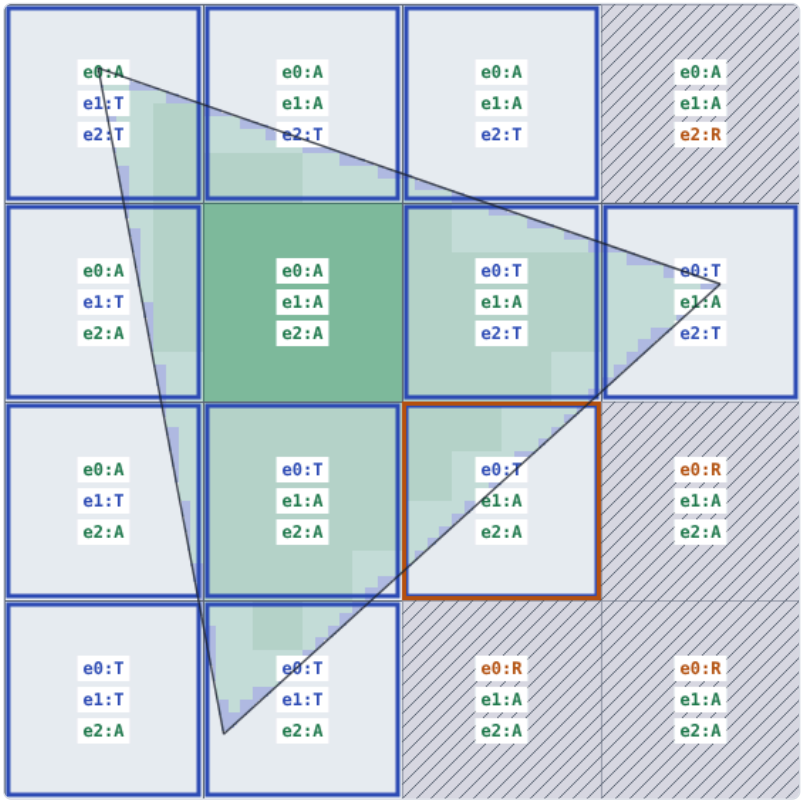
```
Tuile choisie (2, 2), arête e0 :
coin d'acceptation = pixel (191, 191) : E' = −138
360 + 307 062 = 168 702  →  ≥ 0 : l'arête passe
dans la tuile (ou la rejette) : elle reste active
De rejet à acceptation : 307 062 − (0) = 307 062 =
|A|·63 + |B|·63 = 146 097 + 160 965
Tuiles acceptées par e0 : 7 sur 16.
```

À OBSERVER

- On passe du coin de rejet au coin d'acceptation en ajoutant une constante : $|A|\cdot(T-1) + |B|\cdot(T-1)$, le chemin en diagonale à travers la tuile.
- Une tuile peut être acceptée pour une arête et rejetée par une autre. C'est le chapitre suivant.

A7 L'asymétrie, et la tuile pleine

Les deux tests ne jouent pas le même rôle. Un **rejet** par une seule arête suffit à écarter la tuile pour tout le triangle. Une **acceptation** ne concerne qu'une arête : elle dit seulement qu'on peut l'oublier dans cette tuile. Une tuile n'est pleine que si elle est acceptée par les trois arêtes ; on l'émet alors sans aucune rastérisation.




```
Tuiles de la boîte : 11 partielles, 1 pleines, 4
rejetées.

Tuile choisie (2, 2) : e0 traverse, e1 accepte, e2
accepte
→ partielle ; arête(s) active(s) : e0
```

À OBSERVER

- Prenez le triangle « Plein écran » du laboratoire : les 16 tuiles sont pleines. Abrash parle d'une vitesse de rastérisation « approximativement infinie ».
- Dans une tuile partielle, il ne reste souvent qu'une ou deux arêtes actives : tout le travail en dessous en sera réduit d'autant.

Figure A7. Dans chaque tuile, trois lettres, une par arête : **R** rejette, **A** accepte, **T** traverse. Un R suffit à rejeter ; trois A font une tuile pleine ; sinon la tuile est partielle, avec ses arêtes T comme seules arêtes actives.

SCALAIRE

```
constexpr int32_t kTile = 64;
for (int32_t ty = ty0; ty <= ty1; ++ty)          // tuiles de la boîte englobante
for (int32_t tx = tx0; tx <= tx1; ++tx)
{
    const int32_t x0 = tx * kTile, y0 = ty * kTile;   // premier pixel de la tuile
    uint32_t active = 0;                              // bit k = arête k traverse la tuile
    int32_t  eTile[3] = {};
    bool     rejected = false;
    for (int k = 0; k < 3; ++k)
    {
        // 64 bits ICI SEULEMENT : q peut être grand loin de l'arête (étape 5).
        const int64_t e = int64_t(E[k].A) * x0 + int64_t(E[k].B) * y0 + E[k].q;
        if (e + c64[k].rej >= 0) { rejected = true; break; }   // tuile entière dehors
        if (e + c64[k].acc >= 0)                               // l'arête traverse la tuile :
        {
            active |= 1u << k;
            eTile[k] = int32_t(e);        // tient sur 32 bits PARCE QU'elle traverse (étape 5)
        }                                 // sinon : arête acceptée, oubliée dans la tuile
    }
    if (rejected)    continue;
    if (active == 0) EmitFullTile(tx, ty);            // « vitesse approximativement infinie »
    else             Bin(tx, ty, tri, active, eTile); // rastérisée plus tard, tuile par tuile
}
```

POURQUOI CE NIVEAU RESTE SCALAIRE

```
// Abrash fait ce niveau en scalaire, et c'est voulu :
//  - 90 % des triangles tiennent dans UNE tuile : un test de boîte suffit ;
//  - les autres coûtent quelques tuiles chacun, négligeable devant leur shading ;
//  - c'est le seul niveau en 64 bits (q peut être grand loin de l'arête).
// Pour un très grand triangle, on pourrait réutiliser la hiérarchie
// (16 tuiles à la fois) : Abrash le note, sans en avoir eu besoin.
```

A8 Descendre : juger 16 blocs d'un coup

Dans une tuile partielle, on recommence le même raisonnement sur ses 16 blocs de 16 × 16. Pour chaque bloc, on veut la valeur de l'arête à son coin de rejet et à son coin d'acceptation. Inutile de les recalculer : depuis le coin de la tuile, il suffit d'ajouter un pas précalculé. Seize pas, seize additions, **une seule instruction vectorielle**.

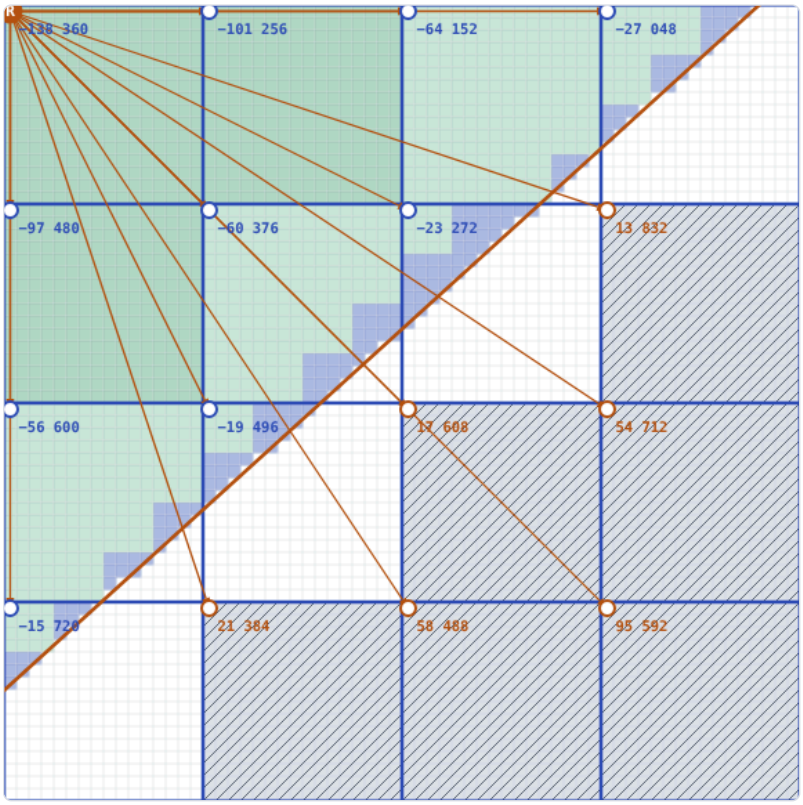


Figure A8. La flèche part du coin de la tuile (rouge) et arrive au coin de chaque bloc (point blanc, avec sa valeur). Chaque flèche est une addition. Blocs grisés : rejetés par cette arête ; teintés en vert (mode acceptation) : acceptés.

Arête :

Coin :

```
Tuile (2, 2) : premier pixel (128, 128), arête e0,
E' = −138 360
Coin de rejet du parent (point rouge) : −138 360

Pas vers le bloc i = A·16·colonne + B·16·rangée
(A·16 = 37 104, B·16 = 40 880)
plus le décalage de coin du bloc : 0
  bloc 0 : −138 360 + 0 = −138 360
  bloc 5 : −138 360 + 77 984 = −60 376
  bloc 15 : −138 360 + 233 952 = 95 592

Blocs rejetés par e0 (valeur ≥ 0) : 6 sur 16
(Le code part du premier pixel du parent, pas de
son coin : c'est la même somme, la table contient
le pas complet.)
```

À OBSERVER

- Chaque pas vaut A·Δx + B·Δy : il ne dépend que de la position relative du bloc dans la tuile, jamais de la tuile elle-même. La même table de 16 pas sert pour toutes les tuiles.
- Les multiplications sont faites une fois par triangle, en préparant les tables. Pendant la descente, il n'y a que des additions.

voie	0 lo	1	2	3	4	5	6	7	8 hi	9	10	11	12	13	14	15
e0 step	0	37 104	74 208	111 312	40 880	77 984	115 088	152 192	81 760	118 864	155 968	193 072	122 640	159 744	196 848	233 952
e0 rej	0	37 104	74 208	111 312	40 880	77 984	115 088	152 192	81 760	118 864	155 968	193 072	122 640	159 744	196 848	233 952
e0 acc	73 110	110 214	147 318	184 422	113 990	151 094	188 198	225 302	154 870	191 974	229 078	266 182	195 750	232 854	269 958	307 062
e1 step	0	−54 912	−109 824	−164 736	10 336	−44 576	−99 488	−154 400	20 672	−34 240	−89 152	−144 064	31 008	−23 904	−78 816	−133 728
e1 rej	−51 480	−106 392	−161 304	−216 216	−41 144	−96 056	−150 968	−205 880	−30 808	−85 720	−140 632	−195 544	−20 472	−75 384	−130 296	−185 208
e1 acc	9 690	−45 222	−100 134	−155 046	20 026	−34 886	−89 798	−144 710	30 362	−24 550	−79 462	−134 374	40 698	−14 214	−69 126	−124 038
e2 step	0	17 808	35 616	53 424	−51 216	−33 408	−15 600	2 208	−102 432	−84 624	−66 816	−49 008	−153 648	−135 840	−118 032	−100 224
e2 rej	−48 015	−30 207	−12 399	5 409	−99 231	−81 423	−63 615	−45 807	−150 447	−132 639	−114 831	−97 023	−201 663	−183 855	−166 047	−148 239
e2 acc	16 695	34 503	52 311	70 119	−34 521	−16 713	1 095	18 903	−85 737	−67 929	−50 121	−32 313	−136 953	−119 145	−101 337	−83 529

Voies 0 à 7 = registre lo (blocs des rangées 0 et 1), 8 à 15 = registre hi (rangées 2 et 3). Bloc i = (colonne i & 3, rangée i >> 2).

Valeurs des 16 blocs 16×16 de la tuile choisie (e + table), puis ET et masques :

voie	0 lo	1	2	3	4	5	6	7	8 hi	9	10	11	12	13	14	15
e0 + rej	−138 360	−101 256	−64 152	−27 048	−97 480	−60 376	−23 272	13 832	−56 600	−19 496	17 608	54 712	−15 720	21 384	58 488	95 592
e0 + acc	−65 250	−28 146	8 958	46 062	−24 370	12 734	49 838	86 942	16 510	53 614	90 718	127 822	57 390	94 494	131 598	168 702
ET des rej	−138 360	−101 256	−64 152	−27 048	−97 480	−60 376	−23 272	13 832	−56 600	−19 496	17 608	54 712	−15 720	21 384	58 488	95 592
→ touched	1	1	1	1	1	1	1	0	1	1	0	0	1	0	0	0
ET des acc	−65 250	−28 146	8 958	46 062	−24 370	12 734	49 838	86 942	16 510	53 614	90 718	127 822	57 390	94 494	131 598	168 702
→ full	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0

TABLES, SCALAIRE

```
// Table d'UN niveau pour UNE arête : 16 enfants de taille S, enfant i = (i & 3, i >> 2).
struct EdgeLevel16Scalar { int32_t step[16], rej[16], acc[16]; };

inline void BuildLevel(int32_t A, int32_t B, int32_t S, EdgeLevel16Scalar& t) noexcept
{
    const CornerOffsets c = Corners(A, B, S);
    for (int i = 0; i < 16; ++i)
    {
        const int32_t cx = i & 3, cy = i >> 2;
        t.step[i] = A * S * cx + B * S * cy;   // du 1er pixel du parent au 1er pixel de l'enfant
        t.rej[i]  = t.step[i] + c.rej;         // ... puis au coin de rejet de l'enfant
        t.acc[i]  = t.step[i] + c.acc;         // ... ou au coin d'acceptation
    }
}
```

```
// Une fois par triangle : 3 arêtes × 3 niveaux (S = 16, 4, 1).
// Au niveau pixel (S = 1), rej = acc = step : une seule table utile.
```

TABLES, AVX2 (16 BLOCS = 2 REGISTRES)

```
struct alignas(32) EdgeLevel16
{
    __m256i rejLo, rejHi;          // voies 0-7 = enfants des rangées 0-1, 8-15 = rangées 2-3
    __m256i accLo, accHi;
    alignas(32) int32_t step[16];  // relu en SCALAIRE après tzcnt (étape 10)
};

inline void BuildLevel16(int32_t A, int32_t B, int32_t S, EdgeLevel16& t) noexcept
{
    const __m256i cx   = _mm256_setr_epi32(0, 1, 2, 3, 0, 1, 2, 3);   // colonne de l'enfant
    const __m256i cyLo = _mm256_setr_epi32(0, 0, 0, 0, 1, 1, 1, 1);   // rangée (registre lo)
    const __m256i cyHi = _mm256_setr_epi32(2, 2, 2, 2, 3, 3, 3, 3);   // rangée (registre hi)
    const __m256i aS = _mm256_set1_epi32(A * S);                       // pas d'un enfant en x
    const __m256i bS = _mm256_set1_epi32(B * S);                       // pas d'un enfant en y

    // Les SEULES multiplications de toute la méthode (vpmulld) : une fois par triangle.
    const __m256i sx = _mm256_mullo_epi32(cx, aS);                          // A·S·cx
    const __m256i lo = _mm256_add_epi32(sx, _mm256_mullo_epi32(cyLo, bS));  // + B·S·cy
    const __m256i hi = _mm256_add_epi32(sx, _mm256_mullo_epi32(cyHi, bS));
    _mm256_store_si256(reinterpret_cast<__m256i*>(t.step),     lo);
    _mm256_store_si256(reinterpret_cast<__m256i*>(t.step + 8), hi);

    const CornerOffsets c = Corners(A, B, S);
    const __m256i rej = _mm256_set1_epi32(c.rej), acc = _mm256_set1_epi32(c.acc);
    t.rejLo = _mm256_add_epi32(lo, rej);  t.rejHi = _mm256_add_epi32(hi, rej);
    t.accLo = _mm256_add_epi32(lo, acc);  t.accHi = _mm256_add_epi32(hi, acc);
}
```

CLASSEMENT, SCALAIRE

```
struct ChildMasks { uint32_t touched, full; };   // bit i = enfant i

inline ChildMasks Classify16Scalar(const int32_t e[3], const EdgeLevel16Scalar* const t[3],
                                   uint32_t active) noexcept
{
    ChildMasks m{ 0xFFFFu, 0xFFFFu };
    for (int k = 0; k < 3; ++k)
    {
        if (!(active & (1u << k))) continue;               // arête acceptée plus haut
        for (int i = 0; i < 16; ++i)
        {
            if (e[k] + t[k]->rej[i] >= 0) m.touched &= ~(1u << i);   // rejeté par k
            if (e[k] + t[k]->acc[i] >= 0) m.full    &= ~(1u << i);   // pas plein pour k
        }
    }
    return m;   // 3 × 16 × 2 = 96 additions et 96 comparaisons, 96 branches
}
```

CLASSEMENT, AVX2

```
// Bit de signe de 16 valeurs (2 registres) → masque de 16 bits.
LV3_FORCEINLINE uint32_t SignMask16(__m256i lo, __m256i hi) noexcept
{
    return  uint32_t(_mm256_movemask_ps(_mm256_castsi256_ps(lo)))          // bits 0-7
         | (uint32_t(_mm256_movemask_ps(_mm256_castsi256_ps(hi))) << 8);   // bits 8-15
}

LV3_FORCEINLINE ChildMasks Classify16(const int32_t e[3], const EdgeLevel16* const t[3],
                                      uint32_t active) noexcept
{
    // −1 : tous les bits à 1, donc « négatif » : élément neutre du ET.
    __m256i rLo = _mm256_set1_epi32(-1), rHi = rLo, aLo = rLo, aHi = rLo;
    for (int k = 0; k < 3; ++k)                        // déroulé par le compilateur
    {
        if (!(active & (1u << k))) continue;           // branche prévisible (même tuile)
        const __m256i b = _mm256_set1_epi32(e[k]);     // valeur au 1er pixel du parent, ×8
        rLo = _mm256_and_si256(rLo, _mm256_add_epi32(b, t[k]->rejLo));  // coins de rejet 0-7
        rHi = _mm256_and_si256(rHi, _mm256_add_epi32(b, t[k]->rejHi));  // coins de rejet 8-15
        aLo = _mm256_and_si256(aLo, _mm256_add_epi32(b, t[k]->accLo));  // coins d'accept. 0-7
        aHi = _mm256_and_si256(aHi, _mm256_add_epi32(b, t[k]->accHi));  // coins d'accept. 8-15
    }
    // Signe du ET = ET des signes : 1 <=> négatif pour TOUTES les arêtes actives.
    return { SignMask16(rLo, rHi), SignMask16(aLo, aHi) };
    // Par arête active : 1 diffusion + 4 vpaddd + 4 vpand. Puis 4 vmovmskps. Aucune comparaison.
}
```

A9 Du vecteur au masque, du masque aux blocs

Les 16 valeurs sont comparées à zéro : on obtient 16 bits, un par bloc, pour chaque arête. Un ET entre les arêtes donne le masque composite : 1 là où le bloc passe le test pour toutes les arêtes. Il reste à visiter les bits à 1, et eux seuls : c'est le passage du parallèle au séquentiel.

Test :

	0	1	2	3	4	5	6	7	8	9	10	11
e0 : valeurs au coin	−65 250	−28 146	8 958	46 062	−24 370	12 734	49 838	86 942	16 510	53 614	90 718	127 822
e0 : < 0 ?	1	1	0	0	1	0	0	0	0	0	0	0
e1 : valeurs au coin (acceptée par la tuile)	−257 603	−312 515	−367 427	−422 339	−247 267	−302 179	−357 091	−412 003	−236 931	−291 843	−346 755	−401 667
e1 : < 0 ?	1	1	1	1	1	1	1	1	1	1	1	1
e2 : valeurs au coin (acceptée par la tuile)	−219 331	−201 523	−183 715	−165 907	−270 547	−252 739	−234 931	−217 123	−321 763	−303 955	−286 147	−268 339
e2 : < 0 ?	1	1	1	1	1	1	1	1	1	1	1	1
ET des 3 arêtes	1	1	0	0	1	0	0	0	0	0	0	0

blocs pleins (masque)

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

masque = 0000 0000 0001 0011
Appuyez sur « Bit suivant ».

À OBSERVER

- Le ET se fait « gratuitement » : une instruction par arête, sur 16 blocs.
- Chaque bit à 1 du masque d'acceptation est un bloc émis tel quel, sans descendre.
- Larrabee avait une instruction faite pour ça, `bsfi` (« trouver le bit suivant à partir de… »). En x64, `tzcnt` trouve le bit le plus bas et `blsr` l'efface : la boucle fait exactement autant de tours que de bits à 1.

LARRABEE, PUIS LA BOUCLE NAÏVE

```
// Larrabee (listing One d'Abrash) :
//     kmov eax, k1        ; masque → registre général
//     bsf  ecx, eax       ; premier bit à 1
//     jnz  ...
// Loop:
//     ...                 ; émettre le bloc ecx
//     bsfi ecx, eax       ; bit SUIVANT, à partir de ecx (instruction ajoutée pour ça)
//     jnz  Loop
//
// Équivalent scalaire portable :
for (int i = 0; i < 16; ++i)
    if (full & (1u << i)) EmitFullBlock(i);   // 16 tours, 16 branches, même si 1 seul bit
```

X64 + BMI1

```
const ChildMasks m = Classify16(e, tab16, active);
const uint32_t partial = m.touched & ~m.full;            // full ⊂ touched (voir Questions)

// _tzcnt_u32 : index du bit 1 le plus bas. _blsr_u32 : efface ce bit (x & (x − 1)).
// La boucle fait EXACTEMENT popcount(m) tours.
for (uint32_t b = m.full; b != 0; b = _blsr_u32(b))
{
    const uint32_t i = _tzcnt_u32(b);
    EmitFullBlock(x0 + 16 * int32_t(i & 3), y0 + 16 * int32_t(i >> 2), 16);
}
for (uint32_t b = partial; b != 0; b = _blsr_u32(b))
{
    const uint32_t i = _tzcnt_u32(b);
    int32_t ce[3];
    for (int k = 0; k < 3; ++k) ce[k] = e[k] + tab16[k]->step[i];   // UNE addition par arête
    Descend4(x0 + 16 * int32_t(i & 3), y0 + 16 * int32_t(i >> 2), ce, active);
}
```

A10 Le niveau suivant : 16 blocs de 4 × 4

Chaque bloc 16 × 16 partiel est traité exactement comme la tuile : 16 enfants de 4 × 4, une table de pas, une addition, un masque. Seules changent la taille des enfants et la table. C'est la force de la méthode : un seul noyau, réutilisé à chaque niveau.

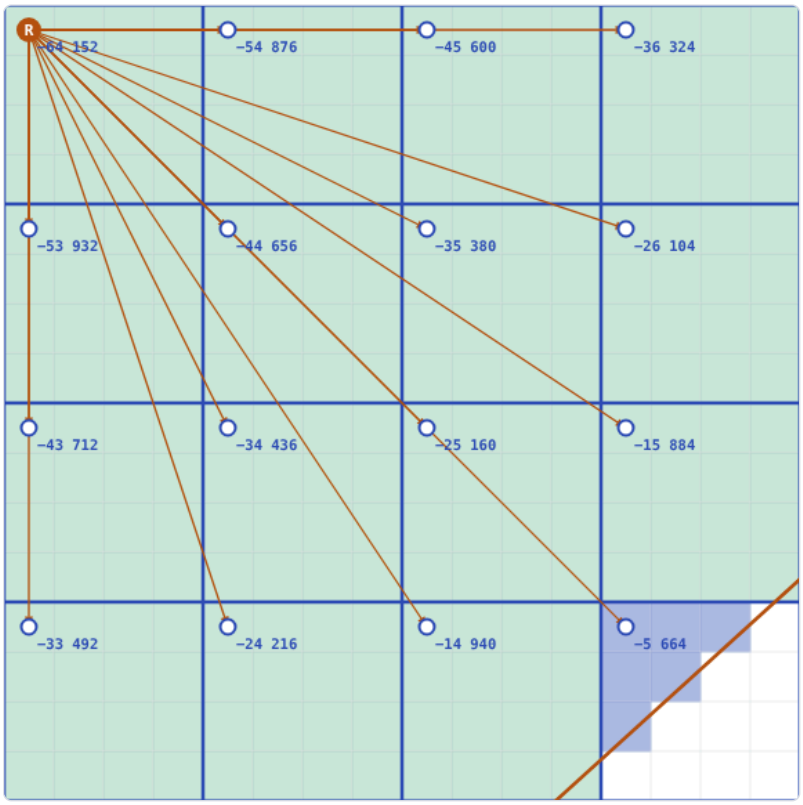


Figure A10. Le bloc 16 × 16 choisi dans le laboratoire, agrandi. Même lecture que la figure A8, un niveau plus bas.

Arête :

Coin :

```
Bloc 16×16 n° 2 (partiel) : premier pixel (160,
128), arête e0, E' = −64 152
Coin de rejet du parent (point rouge) : −64 152

Pas vers le bloc i = A·4·colonne + B·4·rangée
(A·4 = 9 276, B·4 = 10 220)
plus le décalage de coin du bloc : 0
  bloc 0 : −64 152 + 0 = −64 152
  bloc 5 : −64 152 + 19 496 = −44 656
  bloc 15 : −64 152 + 58 488 = −5 664

Blocs rejetés par e0 (valeur ≥ 0) : 0 sur 16
(Le code part du premier pixel du parent, pas de
son coin : c'est la même somme, la table contient
le pas complet.)
```

voie	0 lo	1	2	3	4	5	6	7	8 hi	9	10	11	12	13	14	15
e0 + rej	−64 152	−54 876	−45 600	−36 324	−53 932	−44 656	−35 380	−26 104	−43 712	−34 436	−25 160	−15 884	−33 492	−24 216	−14 940	−5 664
e0 + acc	−49 530	−40 254	−30 978	−21 702	−39 310	−30 034	−20 758	−11 482	−29 090	−19 814	−10 538	−1 262	−18 870	−9 594	−318	8 958
ET des rej	−64 152	−54 876	−45 600	−36 324	−53 932	−44 656	−35 380	−26 104	−43 712	−34 436	−25 160	−15 884	−33 492	−24 216	−14 940	−5 664
→ touched	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ET des acc	−49 530	−40 254	−30 978	−21 702	−39 310	−30 034	−20 758	−11 482	−29 090	−19 814	−10 538	−1 262	−18 870	−9 594	−318	8 958
→ full	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0

APPEL

```
// Dans Descend4 : même classement, tables du niveau S = 4.
const ChildMasks m4 = Classify16(ce, tab4, active);
const uint32_t partial4 = m4.touched & ~m4.full;
// blocs 4×4 pleins : émis ; partiels : étape 12.
```

MÊME FONCTION QU'EN A8

```
// Rien de nouveau : Classify16 de l'étape 9.
// Seules changent les tables (pas = 4 pixels au lieu de 16)
// et les valeurs de départ ce[k] (étape 10).
// C'est la force de la méthode : UN noyau, réutilisé
// à chaque niveau, avec des tables indépendantes
// de la position.
```

A11 Le masque de pixels

Au fond de la hiérarchie, un bloc 4 × 4 partiel. C'est le seul endroit où l'on travaille pixel par pixel : depuis le coin de rejet du bloc, 16 pas mènent aux 16 centres de pixels. Le signe de chaque somme donne un bit ; le ET des arêtes donne le masque de couverture, prêt pour le shading.

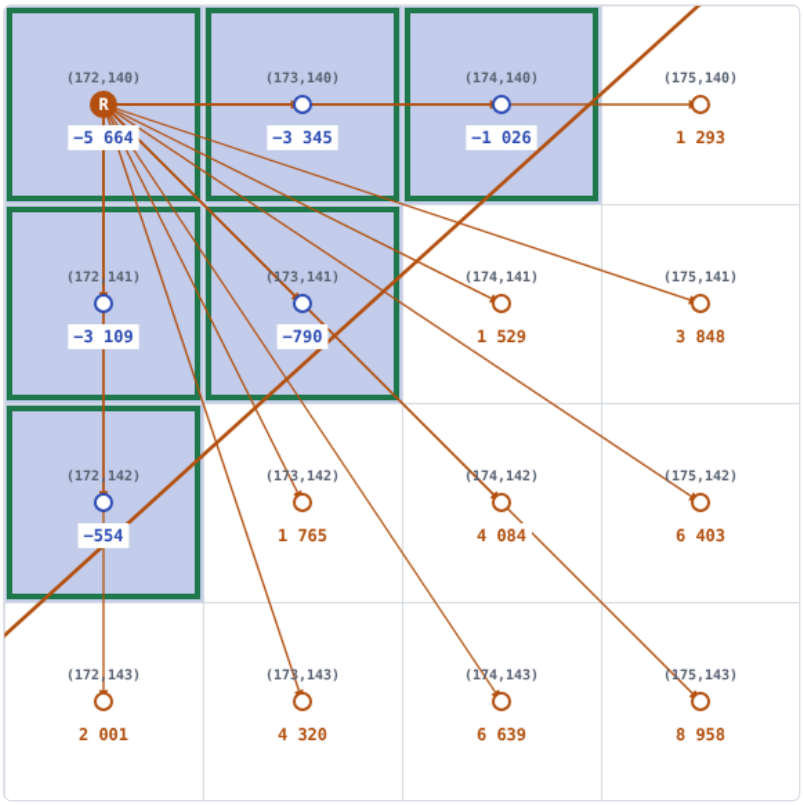


Figure A11. Le bloc 4 × 4 choisi. En bleu : pixels du bon côté de l'arête étudiée. Le masque final (en bas) combine les trois arêtes.

Arête :

```
Bloc 4×4 n° 15 au pixel (172, 140) — partiel
Arête e0 : E' au premier pixel = −5 664
Pas vers le pixel (col, rang) = A·col + B·rang  (A
= 2 319, B = 2 555)

masque e0      = 0000 0001 0011 0111   (bleu
clair)
masque des 3 arêtes = 0000 0001 0011 0111   (cadre
vert) : 6 pixel(s)
(bit 15 à gauche, bit 0 à droite ; bit = 4·rangée
+ colonne)
```

À OBSERVER

- Avec des vecteurs larges, une bonne partie du travail est perdue sur un bloc partiel : beaucoup de voies sont dehors. C'est pourquoi Larrabee s'est arrêté à 16 voies, et pourquoi la hiérarchie ne descend que dans les blocs partiels.
- Abrash range −pas dans la table et teste « e < −pas » : une comparaison au lieu d'une addition et d'une comparaison. En AVX2, on additionne et on lit le bit de signe : même coût.

voie	0 lo	1	2	3	4	5	6	7	8 hi	9	10	11	12	13	14	15
e0 + step	−5 664	−3 345	−1 026	1 293	−3 109	−790	1 529	3 848	−554	1 765	4 084	6 403	2 001	4 320	6 639	8 958
ET (LV3)	−5 664	−3 345	−1 026	1 293	−3 109	−790	1 529	3 848	−554	1 765	4 084	6 403	2 001	4 320	6 639	8 958
bit de signe	1	1	1	0	1	1	0	0	1	0	0	0	0	0	0	0
Abrash : e < −step	1	1	1	0	1	1	0	0	1	0	0	0	0	0	0	0

```
masque = 0000 0001 0011 0111  (6 pixel(s))
```

SCALAIRE

```
// Niveau pixel : S = 1, donc rej = acc = step (le coin EST le centre).
inline uint32_t PixelMask16Scalar(const int32_t e[3], const EdgeLevel16Scalar* const pix[3],
                                  uint32_t active) noexcept
{
    uint32_t mask = 0xFFFFu;
    for (int k = 0; k < 3; ++k)
    {
        if (!(active & (1u << k))) continue;
        for (int p = 0; p < 16; ++p)                // p = 4·py + px dans le bloc 4×4
            if (e[k] + pix[k]->step[p] >= 0) mask &= ~(1u << p);
    }
    return mask;   // bit p = pixel (x0 + (p & 3), y0 + (p >> 2)) couvert
}
```

AVX2

```
LV3_FORCEINLINE uint32_t PixelMask16(const int32_t e[3], const EdgeLevel16* const pix[3],
                                     uint32_t active) noexcept
{
    __m256i lo = _mm256_set1_epi32(-1), hi = lo;     // neutre du ET
    for (int k = 0; k < 3; ++k)
    {
        if (!(active & (1u << k))) continue;
        const __m256i b = _mm256_set1_epi32(e[k]);
        lo = _mm256_and_si256(lo, _mm256_add_epi32(b, pix[k]->rejLo));   // pixels 0-7
        hi = _mm256_and_si256(hi, _mm256_add_epi32(b, pix[k]->rejHi));   // pixels 8-15
    }
    return SignMask16(lo, hi);   // 16 pixels × 3 arêtes : 6 vpaddd, 6 vpand, 2 vmovmskps
}
// Variante Abrash (Larrabee, listing Three) : table NÉGÉE −step, et
//   vcmpgtpi k2{k2}, v_negstep, e   →  « e < −step » : une comparaison par arête.
```

A12 MSAA : un pas de plus

L'anticrénelage multi-échantillon teste plusieurs points par pixel. Pour la méthode, un échantillon n'est qu'un autre point d'arrivée : une table de pas de plus par position d'échantillon, une comparaison de plus par arête sur les blocs 4 × 4 partiels. Les blocs pleins n'ont aucun coût supplémentaire : tous leurs échantillons sont couverts.

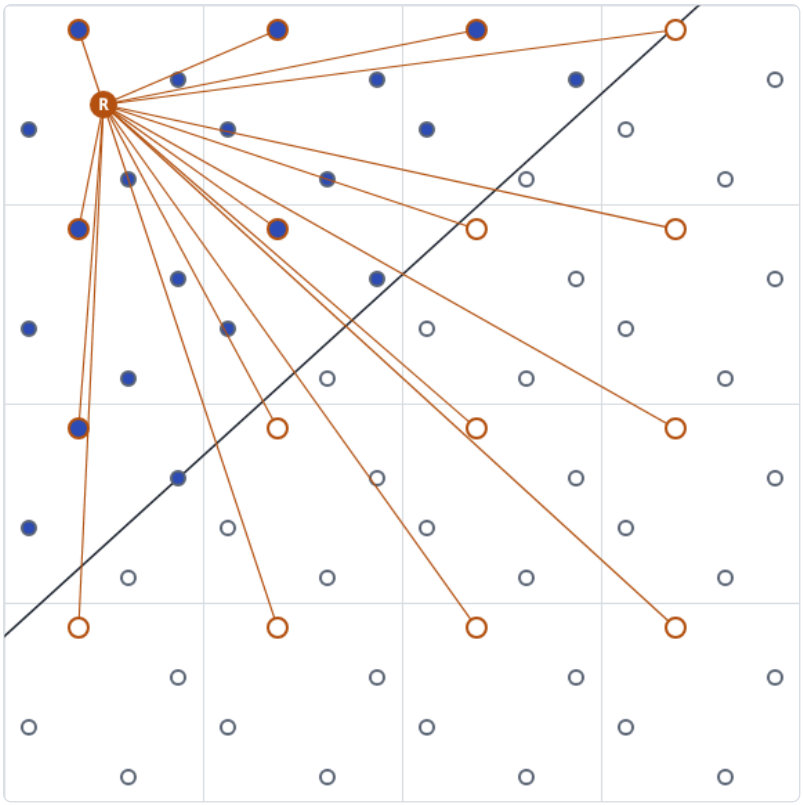


Figure A12. Motif 4× standard (positions en 1/16 de pixel : (−2, −6), (6, −2), (−6, 2), (2, 6)). Les flèches vont vers l'échantillon choisi de chaque pixel.

Échantillon :

```
Bloc 4×4 au pixel (172, 140), quatre échantillons
par pixel :
  éch. 0 : 0000 0001 0011 0111 · 6
  éch. 1 : 0000 0001 0011 0111 · 6
  éch. 2 : 0000 0001 0011 0111 · 6
  éch. 3 : 0000 0000 0001 0011 · 3
  centre : 0000 0001 0011 0111 · 6  (sans MSAA)

Coût : une table, une addition et un ET de plus
par arête et par échantillon, sur les blocs 4×4
PARTIELS seulement.
```

MSAA 4×

```
// Échantillon j du motif MSAA 4× : décalage (dx_j, dy_j) en sous-pixels.
// Table de pas « pixel » propre à l'échantillon, calculée au setup :
//   sampleStep_j[p] = A·(16·px + dx_j) + B·(16·py + dy_j)   (sans la factorisation !)
// Puis, pour un bloc 4×4 partiel : 4 masques au lieu d'un.
for (int j = 0; j < 4; ++j)
    sampleMask[j] = PixelMask16(e, sampleTables[j], active);
// Blocs pleins : couverts pour TOUS les échantillons, aucun coût de plus.
```

A13 Les faux positifs

La hiérarchie n'élimine un bloc que s'il est rejeté par *une* arête. Un bloc proche d'un sommet aigu peut être du bon côté de chacune des trois droites prises séparément, sans toucher le triangle. On y descend alors pour trouver, au fond, un masque vide. Abrash le montre sur ses figures 27 et 29.

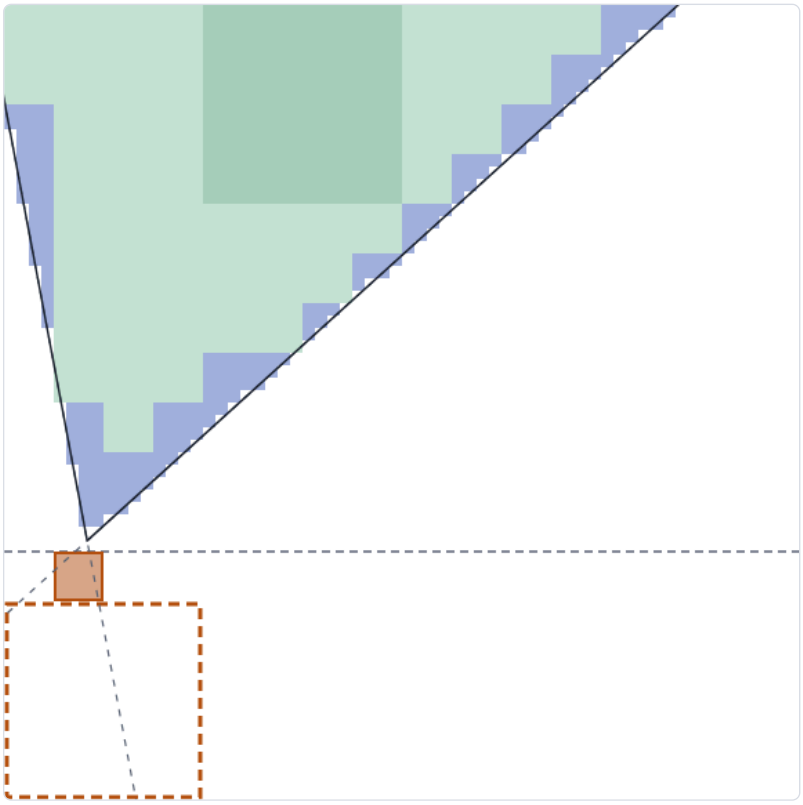


Figure A13. Blocs entourés d'orange : on y est descendu pour rien. Les droites des arêtes sont prolongées : ces blocs sont du bon côté de chacune, mais dans le « coin » formé par deux d'entre elles, hors du triangle.

☐ masque de boîte

```
Tuile montrée : (1, 3), celle qui a le plus de faux positifs.
  blocs 4×4 partiels au masque vide : 1   (orange plein)
  blocs 16×16 partiels sans aucun pixel : 1   (tirets orange)
Sur tout le triangle : 1 bloc(s) 4×4 et 2 bloc(s) 16×16.
Masque de boîte : inactif (pointillés gris : la boîte englobante).
```

À OBSERVER

- Le remède le moins cher : un ET avec le masque des blocs qui croisent la boîte englobante du triangle. Il se calcule en entiers, une fois par niveau.
- Les faux positifs restants sont dans la boîte mais hors du triangle ; on les paie au niveau pixel, et un masque vide ne coûte qu'un saut.

Triangle courant	blocs 16×16 partiels	16×16 sans pixel	blocs 4×4 partiels	4×4 vides	instr. couverture
sans masque de boîte	50	2	154	1	6 500
avec masque de boîte	48	0	153	0	6 421

SANS MASQUE DE BOÎTE

```
const ChildMasks m = Classify16(e, tab, active);
// Un enfant près d'un sommet aigu peut n'être rejeté
// par AUCUNE arête sans toucher le triangle.
// On le visite pour rien, jusqu'à trouver
// un masque de pixels vide.
const uint32_t partial = m.touched & ~m.full;
```

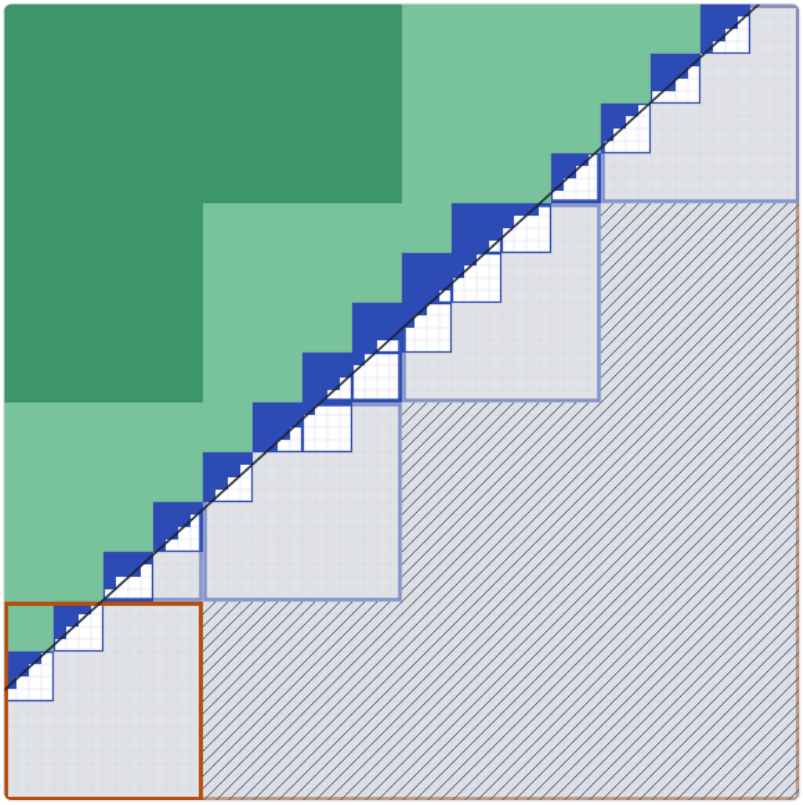
AVEC MASQUE DE BOÎTE

```
// Enfants qui croisent la boîte du triangle : 2 masques de 4 bits
// (colonnes et rangées) calculés en entiers, combinés en 16 bits.
const uint32_t colMask = ColsOverlap(x0, S, bb.x0, bb.x1);   // bit cx
const uint32_t rowMask = RowsOverlap(y0, S, bb.y0, bb.y1);   // bit cy
```

```
const uint32_t boxMask = Expand4x4(colMask, rowMask);        // bit (4·cy + cx)
const uint32_t partial = m.touched & boxMask & ~m.full;
```

A14 Tout ensemble : la descente complète d'une tuile

On rejoue maintenant toute la méthode sur la tuile choisie, dans l'ordre où le moteur l'exécute : juger la tuile, juger ses 16 blocs, émettre les pleins, descendre dans les partiels, juger leurs 16 blocs 4 × 4, puis calculer les masques de pixels. Le compteur suit le travail effectif.



```
Pas 17 / 17
Bloc 16×16 n° 12 : 2 masque(s) de 16 pixels.

Instructions de couverture cumulées (modèle) : 721

Terminé. Pour la même tuile, le chemin C2 jugerait
chaque bloc de 8 de la boîte qui tombe dans la
tuile.
```

Figure A14. Chaque pas de l'animation est une décision du moteur. Vert foncé : bloc 16 × 16 émis plein. Vert clair : bloc 4 × 4 émis plein. Bleu : pixels d'un masque.

Ce que la structure prépare. Hi-Z : un bloc 16 × 16 plein peut être comparé à la profondeur la plus éloignée déjà écrite dans ce bloc et rejeté avant tout shading. **Textures (L09)** : les mipmaps veulent des quads 2 × 2 pour les dérivées d'UV, qu'une émission par blocs 4 × 4 fournit naturellement. **Barycentriques** : les trois valeurs d'arête, divisées par l'aire × 2, sont les coordonnées barycentriques ; z, 1/w et u/w s'avancent avec les mêmes tables (avec la prudence de C1 sur le parenthésage de z).

HI-Z, REVERSE-Z DE LV3

```
// Bloc 16×16 plein : le triangle couvre tout le bloc.
// zNearTri : profondeur la plus proche du triangle (reverse-Z : la plus GRANDE).
// hiZ[b]   : profondeur la plus éloignée déjà écrite dans le bloc (la plus PETITE).
if (zNearTri <= hiZ[b])
    continue;                // tout le bloc est caché : ni shading ni Z-test
EmitFullBlock(b);            // sinon : dessiné ; hiZ[b] mis à jour après
```

PARTIE B · RENDRE LA MÉTHODE EXACTE, PUIS LA TRANSPOSER

B1 La grille sous-pixel : des entiers partout

Les additions de la partie A ne sont exactes qu'en entiers. On accroche donc chaque sommet sur une grille fine : 16 positions par pixel dans ce cours (s = 4), 256 sur Larrabee (s = 8, coordonnées 15.8). C'est le prix d'entrée de la méthode, et c'est aussi ce qui change l'image de LV3 (chapitre B7).

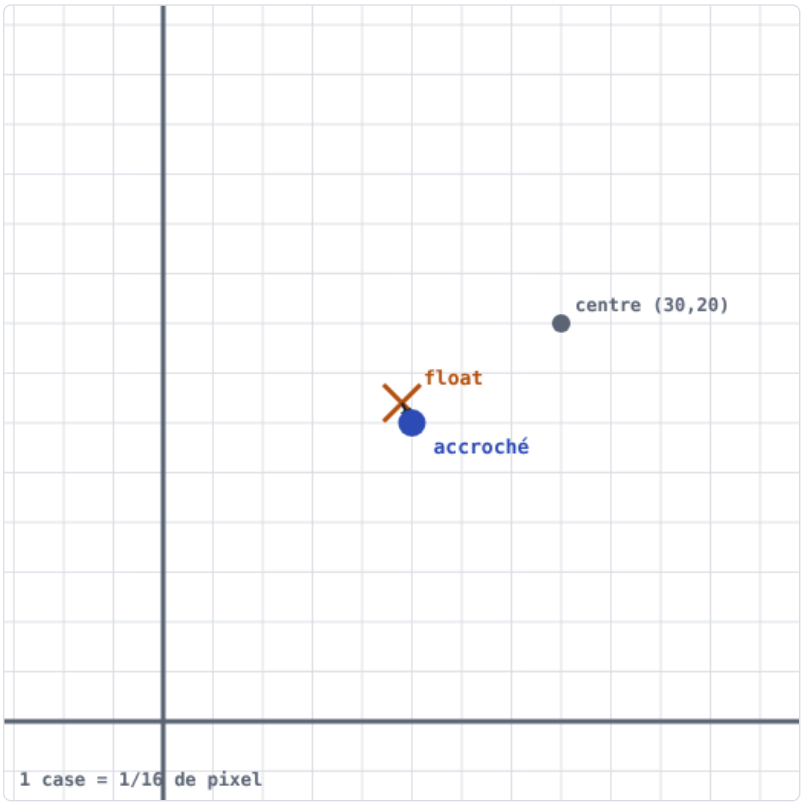


Figure B1. Une fenêtre d'un pixel de côté, centrée sur le sommet choisi. Traits épais : bords de pixels. Croix : position float. Point : position accrochée sur la grille sous-pixel.

Sommet :

```
V0 float  : (30,3000 ; 20,6000) px
× 16      : (484,800 ; 329,600)
arrondi     : (485 ; 330) sous-pixels
soit        : (30,3125 ; 20,6250) px
écart       : (0,0125 ; 0,0250) px, au plus
0,0313
```

À OBSERVER

- L'erreur d'accrochage est au plus d'un demi-sous-pixel : 1/32 de pixel avec s = 4.
- Un sommet partagé par deux triangles donne le même entier dans les deux : c'est la première condition pour qu'il n'y ait ni trou ni doublon.

Sommet	x float	X (entier)	y float	Y (entier)	X / 16, Y / 16 (px)
V0	30,3000	**485**	20,6000	**330**	30,3125 ; 20,6250
V1	230,4000	**3 686**	90,2000	**1 443**	230,3750 ; 90,1875
V2	70,7000	**1 131**	235,1000	**3 762**	70,6875 ; 235,1250

SCALAIRE

```
// x, y : position raster du sommet en pixels (float), sortie de l'étage sommets (B3).
// kSubBits = 4 : 16 sous-pixels par pixel (Larrabee : 8, soit 1/256).
constexpr int32_t kSubBits  = 4;
constexpr float   kSubScale = float(1 << kSubBits);   // 16.0f : puissance de 2, produit EXACT

// std::lrintf : arrondi au plus proche, pair en cas d'égalité (mode par défaut du CPU).
// Surtout PAS int32_t(x * 16) : tronque vers 0, biaise d'un demi sous-pixel,
// et différemment à gauche et à droite de l'origine.
const int32_t X = int32_t(std::lrintf(x * kSubScale));
const int32_t Y = int32_t(std::lrintf(y * kSubScale));
// Garde-bande : |x * 16| < 2^31 est garanti par le clipping (chemin Intersect).
```

AVX2

```
// x8, y8 : abscisses et ordonnées de 8 sommets (SoA, alignés 32 o, comme PositionsSoA).
const __m256  scale = _mm256_set1_ps(16.0f);             // 2^kSubBits, diffusé ×8

// vmulps : exact (puissance de 2). vcvtps2dq : arrondi selon MXCSR = au plus proche, pair
// en cas d'égalité, EXACTEMENT comme std::lrintf. Chaque voie donne le même entier.
const __m256i X8 = _mm256_cvtps_epi32(_mm256_mul_ps(x8, scale));
const __m256i Y8 = _mm256_cvtps_epi32(_mm256_mul_ps(y8, scale));
// Piège : _mm256_cvttps_epi32 (deux t) TRONQUE. Une lettre de plus, un biais de plus.
```

B2 Qui possède un centre posé sur une arête ?

En entiers, un centre de pixel peut tomber *exactement* sur une arête : E = 0. Deux triangles voisins se partagent cette arête. Si les deux le prennent, il est dessiné deux fois ; si aucun ne le prend, c'est un trou. La règle top-left tranche : il appartient au triangle pour lequel l'arête est à gauche ou en haut. En entiers, elle se réduit à retrancher 1 à la constante de ces arêtes.

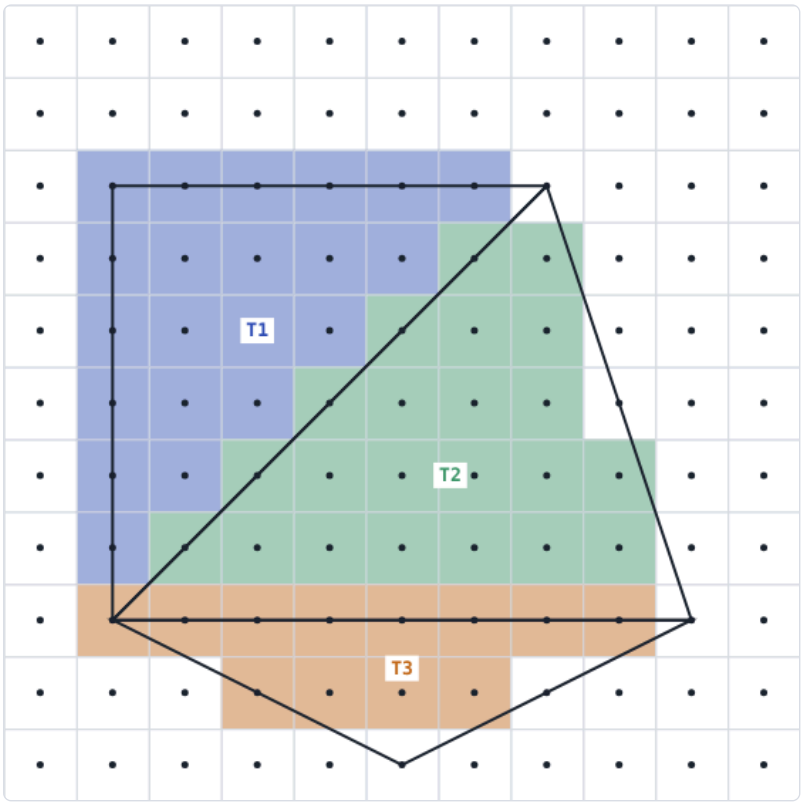


Figure B2. Trois triangles voisins, sommets sur des centres. Les arêtes partagées passent par plusieurs centres. Rouge : dessiné deux fois. Orange : jamais dessiné alors qu'il est à l'intérieur.

Règle :

```
Règle : top-left (biais −1 sur les arêtes gauches et hautes)
Centres dessinés deux fois : 0
Centres intérieurs jamais dessinés : 0
→ chaque centre a exactement un propriétaire.

Arêtes partagées : T1–T2 (diagonale), T2–T3 (horizontale).
Pour T3, l'horizontale est une arête HAUTE : avec top-left, ses centres vont à T3.
```

À OBSERVER

- **E < 0 partout** : les centres posés sur une arête partagée ne sont à personne, ce sont des trous.
- **E ≤ 0 partout** : ils sont à tout le monde, ce sont des doublons (visibles en transparence).
- **Top-left** : chacun a exactement un propriétaire. Gauche : A < 0. Haute : A = 0 et B < 0 (y vers le bas).

Arête du laboratoire	A	B	gauche (A < 0)	haute (A = 0, B < 0)	top-left	C → C'
e0 : V1 → V2	2 319	2 555	non	non	non	−12 234 699 → −12 234 699
e1 : V2 → V0	−3 432	646	oui	non	oui	1 451 340 → 1 451 339
e2 : V0 → V1	1 113	−3 201	non	non	non	516 525 → 516 525

SCALAIRE

```
// Normale sortante (A, B) : E croît vers l'extérieur.
//   arête GAUCHE : intérieur à droite  → (A, B) pointe vers −x → A < 0
//   arête HAUTE  : horizontale (A == 0), intérieur en dessous (y vers le bas) → B < 0
const bool topLeft = (e.A < 0) || (e.A == 0 && e.B < 0);

// E entier : « E <= 0 » équivaut à « E − 1 < 0 ».
// Arête top-left : on inclut E == 0 → on teste E − 1 < 0.  Sinon : E < 0.
// Dans les deux cas le test devient « strictement négatif » : le BIT DE SIGNE.
e.C -= topLeft ? 1 : 0;
```

AVX2 SANS BRANCHE

```
const __m256i zero = _mm256_setzero_si256();
const __m256i aNeg = _mm256_cmpgt_epi32(zero, A8);                    // A < 0 → 0xFFFFFFFF
const __m256i top  = _mm256_and_si256(_mm256_cmpeq_epi32(A8, zero),   // A == 0
                                      _mm256_cmpgt_epi32(zero, B8));  //   et B < 0
const __m256i tl8  = _mm256_or_si256(aNeg, top);                      // −1 si top-left, 0 sinon

// Le masque VAUT −1 sur les voies top-left : l'ajouter retranche 1. Aucune branche.
C8 = _mm256_add_epi32(C8, tl8);     // (si C tient sur 32 bits ; sinon même idée en 64 bits)
```

B3 Retirer les bits sous-pixel de la partie qui varie

On ne teste jamais que des centres de pixels, dont la partie fractionnaire vaut toujours ½. La valeur au centre s'écrit donc E = 16·(A·x + B·y) + K, où seul K est constant. On sépare K en un quotient et un reste : K = 16·q + r, avec 0 ≤ r < 16. Le reste ne change jamais le signe ; on travaille avec **E' = A·x + B·y + q**, en pixels entiers. Un pas d'un pixel ajoute A au lieu de 16·A : quatre bits de gagnés.

LA PREUVE EN DEUX LIGNES

AVEC VOTRE TRIANGLE

Si A·x + B·y + q ≤ −1, alors E ≤ −16 + r ≤ −1 : dedans. Si A·x + B·y + q ≥ 0, alors E ≥ r ≥ 0 : dehors. Les signes de E et de E' sont toujours les mêmes.

```
Arête e0, rangée y = 140 : K = −12 195 707 = 16·(−762 232) + 5
E' change de signe au même pixel que E ; chaque pas vaut A = 2 319.
```

voie	0 lo	1	2	3	4	5	6	7	8 hi	9	10	11	12	13	14	15
pixel x	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181
E au centre (×16²)	−313 243	−276 139	−239 035	−201 931	−164 827	−127 723	−90 619	−53 515	−16 411	20 693	57 797	94 901	132 005	169 109	206 213	243 317
16·E' + r	−313 243	−276 139	−239 035	−201 931	−164 827	−127 723	−90 619	−53 515	−16 411	20 693	57 797	94 901	132 005	169 109	206 213	243 317
E' (pixels)	−19 578	−17 259	−14 940	−12 621	−10 302	−7 983	−5 664	−3 345	−1 026	1 293	3 612	5 931	8 250	10 569	12 888	15 207
pas : + A	0	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319	2 319

Figure B3. La rangée du bloc 4 × 4 choisi, prolongée sur 16 pixels. En haut, E au centre (sous-pixels) ; en bas, E' (pixels). Les deux changent de signe au même pixel.

SCALAIRE

```
// Centre du pixel (x, y), en sous-pixels : PX = 16·x + 8, PY = 16·y + 8.
//   E(PX, PY) = A·(16x + 8) + B·(16y + 8) + C'
//             = 16·(A·x + B·y) + K            avec K = 8·(A + B) + C'
// On écrit K = 16·q + r avec 0 <= r < 16 (q = PLANCHER de K / 16).
const int64_t K = int64_t(8) * (e.A + e.B) + e.C;   // e.C déjà biaisé (étape 3)
const int64_t q = K >> kSubBits;   // décalage ARITHMÉTIQUE = plancher (garanti en C++20,
                                   // fait par MSVC sur x64). JAMAIS K / 16 : tronque vers 0.

// Désormais, en pixels ENTIERS :  E'(x, y) = A·x + B·y + q
//   +1 pixel en x : + A      +1 ligne : + B
// et E(centre) < 0  <=>  E'(x, y) < 0   (preuve ci-contre)
```

LA PREUVE

```
E(centre) = 16·(A·x + B·y + q) + r,   avec 0 <= r < 16

Si A·x + B·y + q <= −1 :
    E <= −16 + r <= −16 + 15 = −1 < 0        → dedans

Si A·x + B·y + q >= 0 :
    E >= 0 + r >= 0                          → dehors

Donc signe(E) = signe(E'), pour TOUT centre de pixel.
Le reste r disparaît : il est le même partout.
```

B4 Pourquoi 32 bits suffisent dans une tuile

Abrash affirme que dans une tuile de 128 × 128 au plus, 32 bits suffisent pour toute arête qui n'est ni rejetée ni acceptée. Le raisonnement tient en une phrase : une arête qui traverse la tuile s'y annule quelque part, et E' ne varie que de (|A| + |B|) × T sur la tuile ; donc |E'| ≤ (|A| + |B|) × T partout dans la tuile. Le calcul au coin de la tuile se fait une fois en 64 bits, la descente en 32.

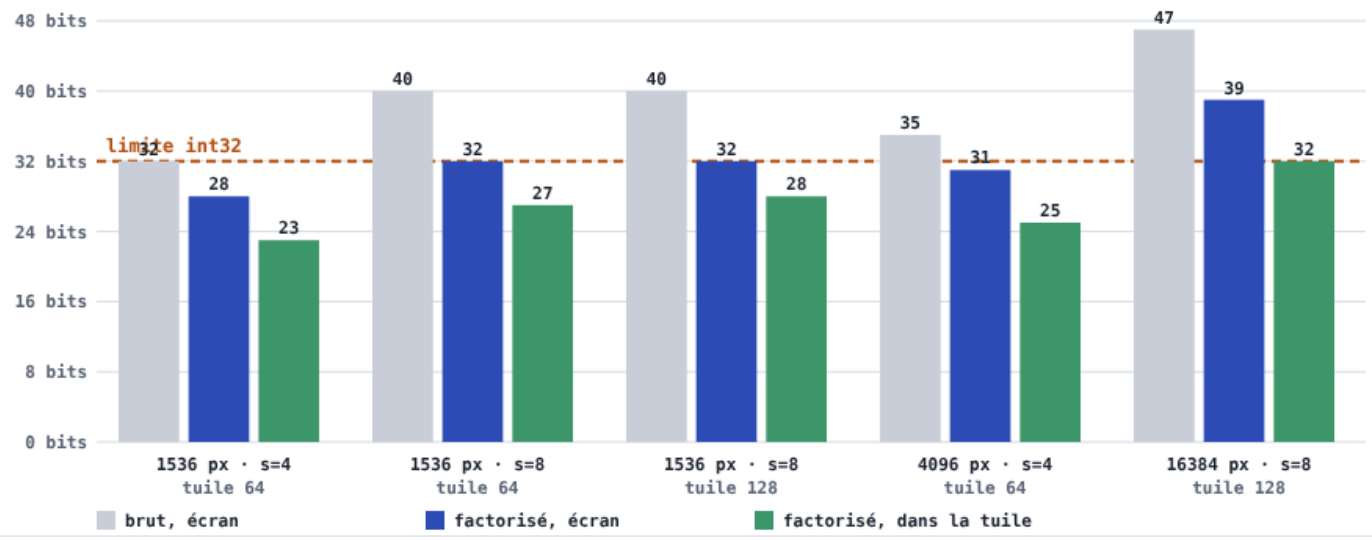


Figure B4. Bits signés nécessaires, pour cinq configurations. Trois barres : l'équation brute sur tout l'écran, l'équation factorisée (B3) sur tout l'écran, l'équation factorisée dans une tuile traversée par l'arête. La ligne marque 32 bits.

```
Votre triangle : |A|, |B| max = 3 432 ; plus grand |E'| d'une arête active à l'entrée d'une tuile = 301 880 (20 bits signés).
```

B5 Contre-exemples chiffrés

Quatre « simplifications » qui semblent anodines. Chacune est mesurée ici avec le même code que le reste du cours, sur des éventails de 8 triangles qui partagent leurs arêtes : chaque pixel intérieur doit y être dessiné exactement une fois.

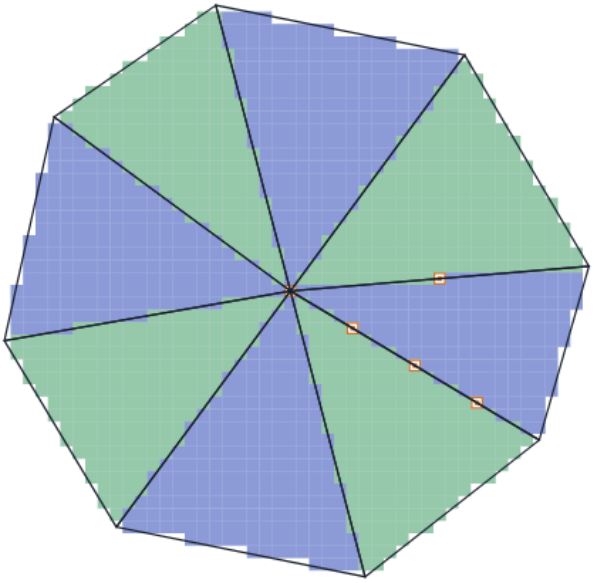


Figure B5. Un éventail, sommets sur des centres. Deux teintes alternées pour les 8 triangles. Rouge : doublon. Orange : trou.

Variante :

```
Éventail n° 7, variante « fixe, E < 0 » :
  doublons : 0   (rouge)
  trous    : 5   (orange cerclé)
→ couture : les arêtes partagées ne sont pas partagées de façon cohérente.
```

Sur 300 éventails

Variante (300 éventails)	Sommets quelconques : doublons / trous / éventails touchés	Sommets sur des centres : doublons / trous / éventails touchés
Float direct (C2, LV3)	0 / 0 / 0	0 / 0 / 0
Float incrémental, amorcé par ligne	0 / 0 / 0	0 / 0 / 0
Float incrémental, amorcé une fois	0 / 0 / 0	0 / 0 / 0
Virgule fixe + top-left (Abrash)	0 / 0 / 0	0 / 0 / 0
Virgule fixe, E < 0 partout	0 / 32 / 16	0 / 4108 / 300
Virgule fixe, E ≤ 0 partout	49 / 0 / 26	8308 / 0 / 300
Virgule fixe, K / 16 au lieu de K >> 4	0 / 211 / 151	0 / 868 / 244

K / 16 au lieu de K >> 4

La division entière C++ tronque vers zéro : pour K négatif, q est trop grand de 1 et des pixels intérieurs passent dehors. Le décalage arithmétique donne le plancher.

```
Triangle du laboratoire : q tronqué ≠ q plancher sur 1 arête(s) ; 0 pixel(s) changent de décision.
  e0 : K = −12 195 707 → K >> 4 = −762 232, mais K / 16 = −762 231 (tronqué vers 0)
Population (300 triangles, s = 4) : 22 pixels changent, sur 22 triangles. Toujours dans le même sens : des pixels intérieurs perdus.
```

Avancer en float par additions

C'est précisément ce que la règle de LV3 interdit (« jamais d'arêtes incrémentales en float »). La virgule fixe rend la même addition exacte : c'est elle qui autorise la méthode d'Abrash.

```
Éventails calculés ici : incrémental amorcé par ligne 0 doublons / 0 trous ; amorcé une fois 0 / 0.
Population (300 triangles en coordonnées d'écran ≈ 1024 à 1280 px) : 1 décision(s) changée(s) (amorce par ligne), 1 (amorce unique), sur 318 780 pixels testés.
Référence C0 (hors moteur, 21/10) : 273 décisions sur 207 M pixels ; 145 doublons et 161 trous sur 2 999 éventails.
Les éventails de ce tirage n'en montrent aucune : l'erreur touche environ un pixel sur un million. Le bouton « grands éventails » en fait apparaître.
```

Un int32_t trop tôt

```
Écran de 1536 px, s = 8, arête (2 ; 3) → (1500,5 ; 899) px : A = −229 376, B = 383 616, C = −177 176 576
pixel (700, 420) : E exact = −15 220 736 (dedans)   E en int32 = −15 220 736 (dedans)   ✓
pixel (0, 899) : E exact = 88 129 486 848 (dehors)   E en int32 = −2 064 826 368 (dedans)   ✗ SIGNE FAUX

Près de l'arête, l'arithmétique modulo 2^32 retombe juste : les débordements intermédiaires s'annulent quand le RÉSULTAT tient sur 32 bits.
Loin de l'arête, le résultat lui-même ne tient plus : le signe est faux, une tuile lointaine serait acceptée.
D'où le niveau tuile en 64 bits, et seulement les arêtes actives en int32 (B4).
```

B6 Ce que coûte la méthode, comparée à C2

La hiérarchie paie un prix fixe (préparer les tables, juger les tuiles) pour économiser ensuite sur l'intérieur. Elle ne gagne que si l'intérieur est assez grand. Le modèle ci-dessous compte les instructions de couverture seulement ; c'est un ordre de grandeur, pas une mesure.

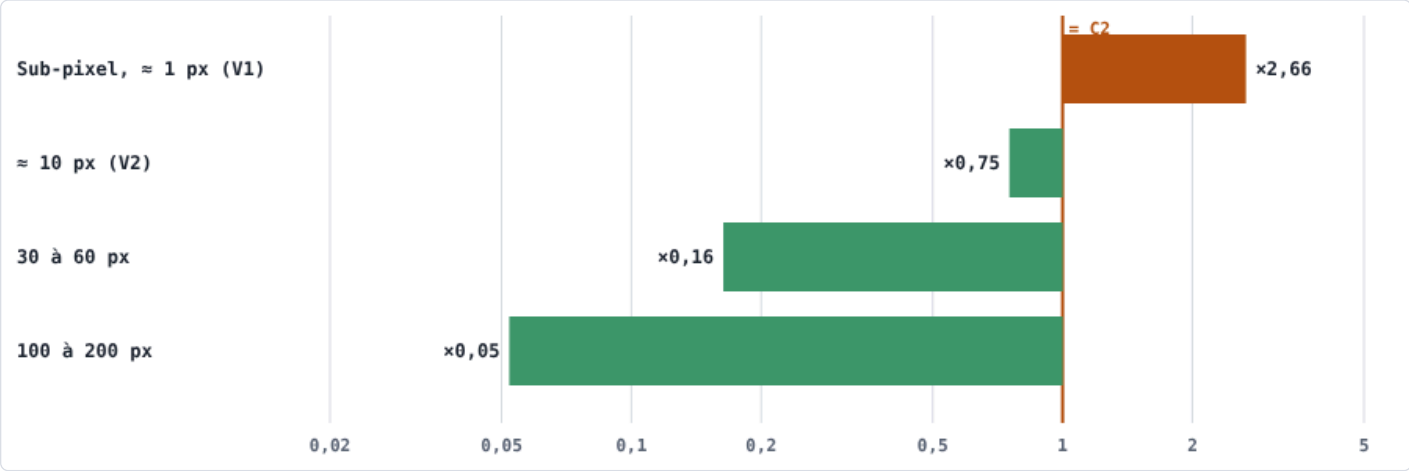


Figure B6. Instructions de couverture de la hiérarchie divisées par celles de C2, par classe de taille de triangle. Sous la ligne 1, la hiérarchie coûte moins.

Triangle du laboratoire	Hiérarchie	C2 (blocs de 8)
Instructions de couverture (modèle)	**6 500**	**207 826**
dont setup (sommets, arêtes, tables)	130	34
Pixels couverts / émis sans test	20 053 / 18 848	20 053 / 0

Modèle : hiérarchie = 130 de setup, 18 par tuile, 7 + 9 par arête active pour juger 16 blocs, 4 + 5 par arête active pour un masque de pixels, 3 par bit parcouru plus une addition par arête pour descendre, 3 par bloc émis. C2 = 34 par triangle, 37 par bloc de 8 de la boîte. Le shading n'est compté ni d'un côté ni de l'autre.

Ce que dit le modèle. Pour un triangle d'un pixel (V1), la hiérarchie coûte environ 2,7 fois C2 : 130 instructions de setup pour rien ; c'est le chemin C3 qu'il faut à V1. Vers 10 pixels (V2), elle est à 0,75 : aucun bloc 16×16 n'est plein, il n'y a que des blocs 4×4 partiels. Elle ne rapporte vraiment qu'à partir de quelques dizaines de pixels, quand les blocs pleins prennent le relais. D'où les chemins adaptatifs d'Abrash, et la règle du jalon D : construire une scène de grands triangles avant de juger.

B7 Transposer dans LV3 : deux voies, une règle

LV3 a trois règles qui contraignent toute transposition : l'image et le Z-buffer doivent rester identiques au bit (TNR), jamais d'arêtes incrémentales en float, et les valeurs d'arête de C2 servent aussi de barycentriques pour z. D'où deux voies.

Voie	Ce qu'elle fait	Image	Gain attendu dans LV3
D-a virgule fixe	Abrash à la lettre : partie A sur des entiers (B1 à B4).	`change` accrochage sous-pixel ; nouvelles références TNR	Blocs pleins sans test, masques exacts, MSAA et quads. Mais les barycentriques de z sont à recalculer, alors que C2 les obtenait avec les arêtes.
D-b pré-filtre prudent	Hiérarchie au-dessus de C2 : la fonction d'arête float est évaluée *directement* aux coins d'un bloc 8 × 8, avec une marge τ. Rejet sûr, plein sûr, sinon C2.	`identique au bit`	Surtout le rejet des blocs vides. Un bloc plein évite les comparaisons et le masque, pas les 7 opérations par voie.

Règle écrite avant la mesure (plan de la phase G). À la clôture de C, construire la scène V3 (grands triangles) et refaire le découpage de C0 sur le binaire C2 + C3. Si les arêtes pèsent au moins 40 % et que Covered / Tight est sous 50 % sur V3, ouvrir D1 en voie D-b ; sinon, le jalon reste rangé. Décider D-a avant L09 ; D2 (tuiles et binning) avec L13.

D-B (CONVENTION LV3 : POSITIF = DEDANS)

```
// Bloc 8×8 : on évalue la fonction d'arête float de C2 (EdgeFunction, 7 opérations)
// DIRECTEMENT aux 4 centres extrêmes du bloc. Jamais par additions (règle C0).
// tau : borne de l'erreur d'arrondi d'EdgeFunction (à démontrer comme ObjBackThreshold, B5).
for each edge k:
    wMax[k] = max(w(c00), w(c70), w(c07), w(c77));   // coin le plus « dedans » (LV3 : positif)
    wMin[k] = min(...);
if (any k: wMax[k] < -tau)  → bloc SÛREMENT vide : sauté, aucun accès mémoire
if (all k: wMin[k] > +tau)  → bloc SÛREMENT plein : masque de couverture = 0xFF par ligne
else                        → ForEachCoveredBlock8 (C2) sur les 8 lignes du bloc
// Les décisions par pixel restent celles de C2 : image identique au bit.
```

GREFFE SUR C2 : UN BLOC 8×8 = 8 APPELS AU SINK

```
// Le bloc 8×8 de la hiérarchie s'aligne sur les blocs de 8 de C2.
// Un bloc plein = 8 appels au sink de C2, cover = toutes voies :
for (int32_t r = 0; r < 8; ++r)
{
    sink.BeginRow(y0 + r);
    // w0..w2 et barycentriques restent calculés par voie (C2) :
    // en LV3, ce sont eux qui donnent z (même parenthésage, C1).
    sink.Block(x0, allLanes, 0xFFu, b0, b1, b2);
}
// Gain réel : les blocs VIDES sautés. Les blocs pleins évitent
// les comparaisons et le masque, pas les 7 opérations par voie.
```

VÉRIFIER Tests hors moteur, filets Debug, prédictions

1. Vérification exécutée ici

Le laboratoire compare en permanence la hiérarchie à une évaluation directe de chaque pixel (triangle courant : identique, 20 053 pixels, 204 propagations contrôlées, 0 écart). Le bouton refait la comparaison sur une population de triangles.

```
300 triangles (tailles 1 à 120 px, graine 2026, s = 4), 104 671 pixels couverts :
  masque hiérarchique ≠ force brute : 0 pixel     ✓
  valeur propagée (e + pas) ≠ évaluation directe : 0     ✓
  mutation K / 16 : 22 pixels changent (sur 22 triangles) : le test la détecte ✓
calculé en 1,0 s
```

2. Test hors moteur à écrire pour D1

TEST_D1_HIERARCHY.CPP (EXE)

```
// Test_D1_Hierarchy.cpp (EXE) — à écrire à l'ouverture de D1.
// 1. Identité : 100 000 triangles tirés comme la TNR C2 (tailles 1 à 400 px, vues
//    317 × 203 et 61 × 180 collée au bord), s = 4. Pour chacun :
//       masque hiérarchique (tuiles → 16×16 → 4×4 → pixels)
//    == évaluation directe A·x + B·y + q < 0 sur chaque pixel de la vue.
```

```
//    Exigence : 0 différence. Compter aussi blocs pleins / partiels / faux positifs.
// 2. Étanchéité : 3 000 éventails de 8 triangles, sommets quelconques puis sur des
//    centres. Exigence : 0 doublon, 0 trou à l'intérieur de l'octogone.
// 3. Mutations (chacune DOIT faire échouer le test) :
//       K / 16 au lieu de K >> 4 ; biais top-left retiré ; S au lieu de S − 1 dans
//       Corners (doit rester EXACT, mais moins de blocs pleins : compteur, pas échec) ;
//       eTile tronqué en int16_t.
// 4. Voie D-b : couleur ET Z-buffer identiques à C2 (kSimdAlways) sur les mêmes triangles.
```

3. Filets Debug (LV3_ASSERT)

- À l'entrée d'une tuile : pour chaque arête active, $|E'| < 2^{30}$ (B4).
- Après chaque descente : la valeur propagée égale l'évaluation directe au premier pixel de l'enfant (une fois sur N, tirage fixe).
- `(full & ~touched) == 0` à chaque niveau.
- Somme des pixels émis (blocs pleins + masques) = `PixelsCovered` du chemin C2 sur la même frame.

4. Prédictions à écrire avant le lot D1

Quand V3 existera, avant toute mesure : blocs 8 × 8 rejetés par frame, part des pixels émis par des blocs pleins, `PixelsTested` attendu, effet sur `Render` pour V1, V2, V3 et L. Règle : compteurs et image identiques, sinon refus ; V1 plus lent de plus de 3 % (runs propres), refus.

5. Liste de contrôle du cours

- [ ] Je sais placer le coin de rejet d'une arête à partir des signes de A et B, sans calcul.
- [ ] Je sais dire pourquoi un rejet vaut pour le triangle et une acceptation pour une arête.
- [ ] Je sais pourquoi la même table de pas sert pour toutes les tuiles.
- [ ] Je sais pourquoi K >> s et non K / 2^s, et pourquoi 32 bits suffisent dans une tuile.
- [ ] Je sais pourquoi la voie D-b ne change pas l'image, et pourquoi elle rapporte moins que D-a.

QUESTIONS Questions de fond

Pourquoi tester les centres et pas les coins géométriques des blocs, comme Abrash ?

Abrash prend le coin géométrique du bloc (le bord du pixel), puis un pas jusqu'au centre au niveau des pixels. C'est correct, et un peu prudent : un bloc dont tous les centres sont dedans peut avoir un coin géométrique dehors, et il ne sera pas déclaré plein. En prenant directement les centres extrêmes (décalage S − 1 au lieu de S), le test colle exactement à ce qu'on échantillonne. Les deux choix donnent la même image ; celui-ci accepte un peu plus de blocs.

Pourquoi le masque des blocs pleins est-il toujours inclus dans celui des blocs touchés ?

Pour une arête, la valeur au coin d'acceptation est le maximum sur le bloc, celle au coin de rejet le minimum. Si le maximum est négatif, le minimum l'est aussi. Accepté implique donc non rejeté, arête par arête, et le ET conserve l'inclusion.

Pourquoi 16 enfants, et pas 8 pour coller à AVX2 ?

16 = 4 × 4 garde des blocs carrés, ce que veulent le shading (quads) et le cache. Sur Larrabee, c'était exactement un registre. En AVX2, c'est deux registres, sans changer la structure. En AVX-512 (i9, B6), on retrouve un registre et de vrais masques k. Pour LV3 en voie D-b, des blocs de 8 × 8 alignés sur les blocs de 8 de C2 sont plus naturels.